TEK-UP
Ecole Supérieure Privée
Technologie & Ingénierie

Chapitre 5: Le Diagramme de Séquence

Auteur :

Mme Ines AGREBI



Qu’est ce qu’un diagramme de séquence?

- Un diagramme de séquence fait partie des diagrammes comportementaux (dynamique) et plus précisément des diagrammes d’interactions.

- Il possède une dimension temporelle mais ne représente pas explicitement les liens entre les objets.

- Il permet de représenter des échanges entre les différents objets et acteurs du système en fonction du temps.

- A moins que le système à modéliser soit extrêmement simple, nous ne pouvons pas modéliser la dynamique globale du système dans un seul diagramme; nous ferons donc appel à un ensemble de diagrammes de séquences chacun correspondant à une sous fonction du système, généralement pour illustrer un cas d'utilisation.

- Un diagramme de séquence permet de visualiser les messages par une lecture de haut en bas:

➢ L'axe vertical représente le temps

➢ L'axe horizontal représente les objets qui collaborent

➢ Une ligne verticale en pointillé est attachée à chaque objet et représente sa durée de vie,

Exemple

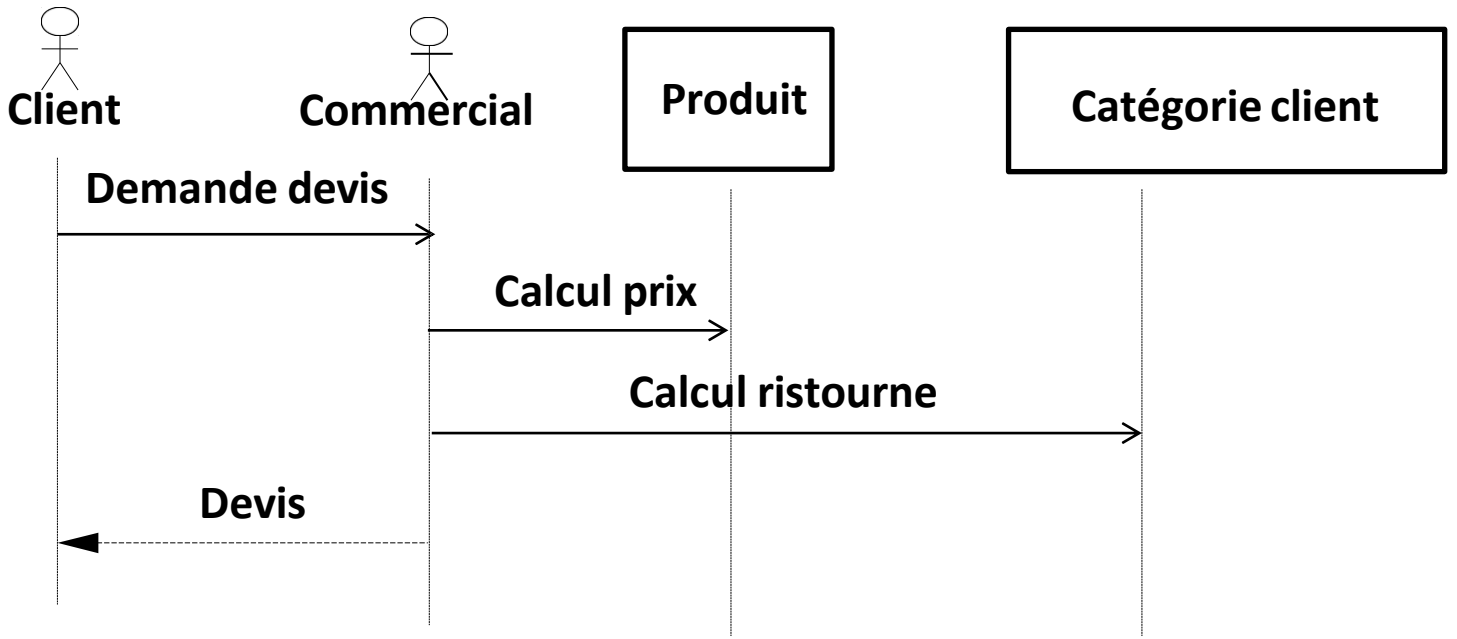
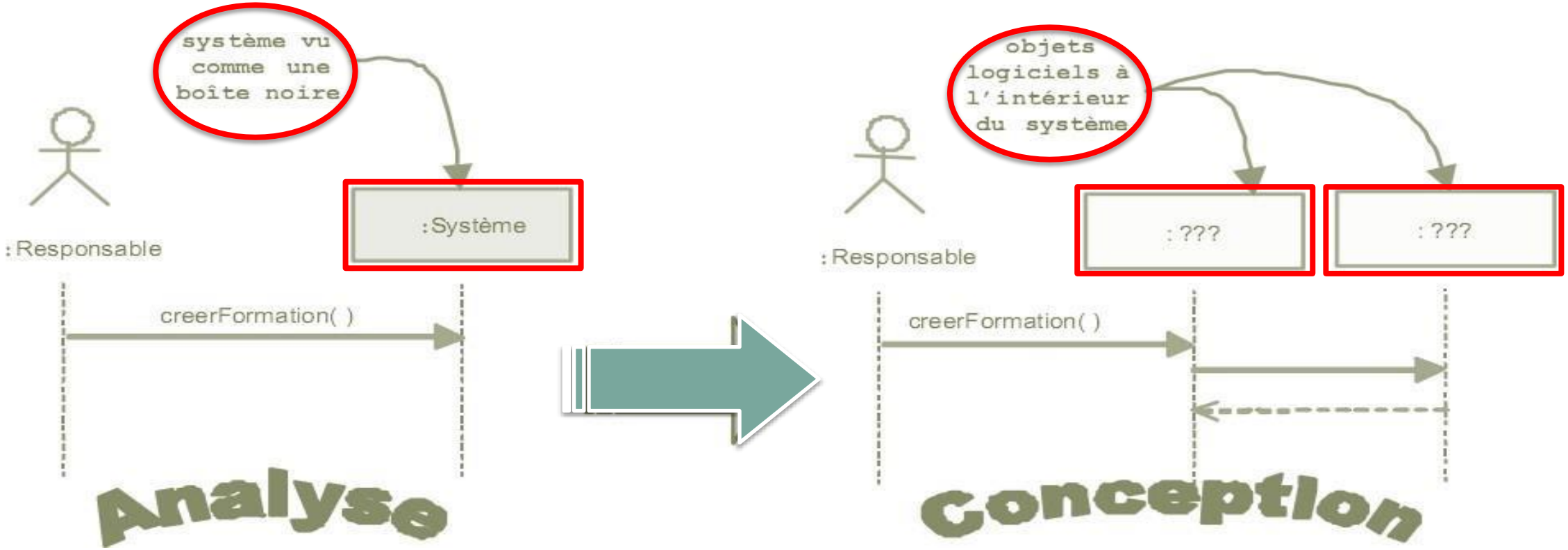

Client
Commercial
Produit
Catégorie client
Demande devis
Calcul prix
Calcul ristourne
Devis

Diagramme de séquence système Vs Diagramme de séquence Objet



Objectifs du diagramme de séquence Système:

- Montrer les interactions ***des acteurs externes*** (obtenus avec le diagramme des UC) avec le système.
- Sans détailler les objets qui participent aux système (système= boite noire, vision Utilisateur)
- Vision globale de tout ce que le système peut faire
- Usages

- Définir les interactions entre les différents composants logiciels

Objectifs du diagramme de séquence Objet:

- Représenter les interactions entre un acteur et les objets (vision Interne).
- Modélisation des scénarios Utilisateur
- Vision temporelle d'une interaction
- Chaque objet est symbolisé par une barre verticale
- Le temps s'écoule de haut en bas, de sorte que la numérotation des messages est optionnelles.
- Souvent utilisé pour représenter le déroulement d'un cas d'utilisation.

Définition

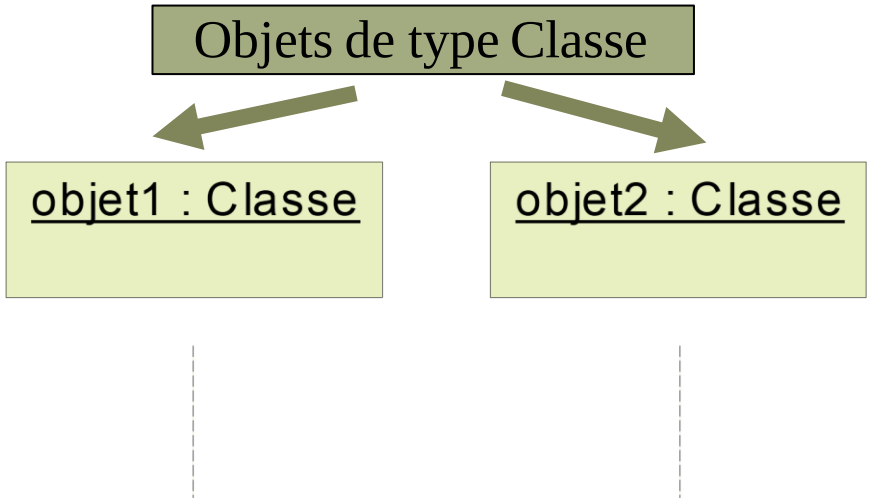
- Les diagrammes de séquences permettent de représenter des collaborations entre des objets pour la réalisation d'un cas d'utilisation, selon un ***point de vue temporel***.

- Dans ces diagrammes on y met l'accent sur la chronologie des envois de messages entre les objets.

Représentation de Scénarios

- Il ya autant de diagrammes de séquence qu'il ya de scénarios.
- Un **Scénario** montre une séquence particulière d'interactions entre objets, dans un seul contexte d'exécution du système.
- Un**scénario** peut être vu comme une réponse à un besoin ou une partie d'un besoin du diagramme des **Uses Cases**.
- On yfait intervenir des **objets**, des **messages** et des **événements**

objets messages événements



Rappel: scénario

- Un cas d'utilisation regroupe une famille de scénarios.
- Un scénario = séquence d'interactions entre le système et ses acteurs.

- On part des ***cas d'utilisation***
- On étudie comment le scénario se déroule à l'intérieur dans la boite noire, comment les classes interagissent dans ce scénario.
- Expose la ***vision utilisateur***: n'impose rien sur les choix d'implémentation.

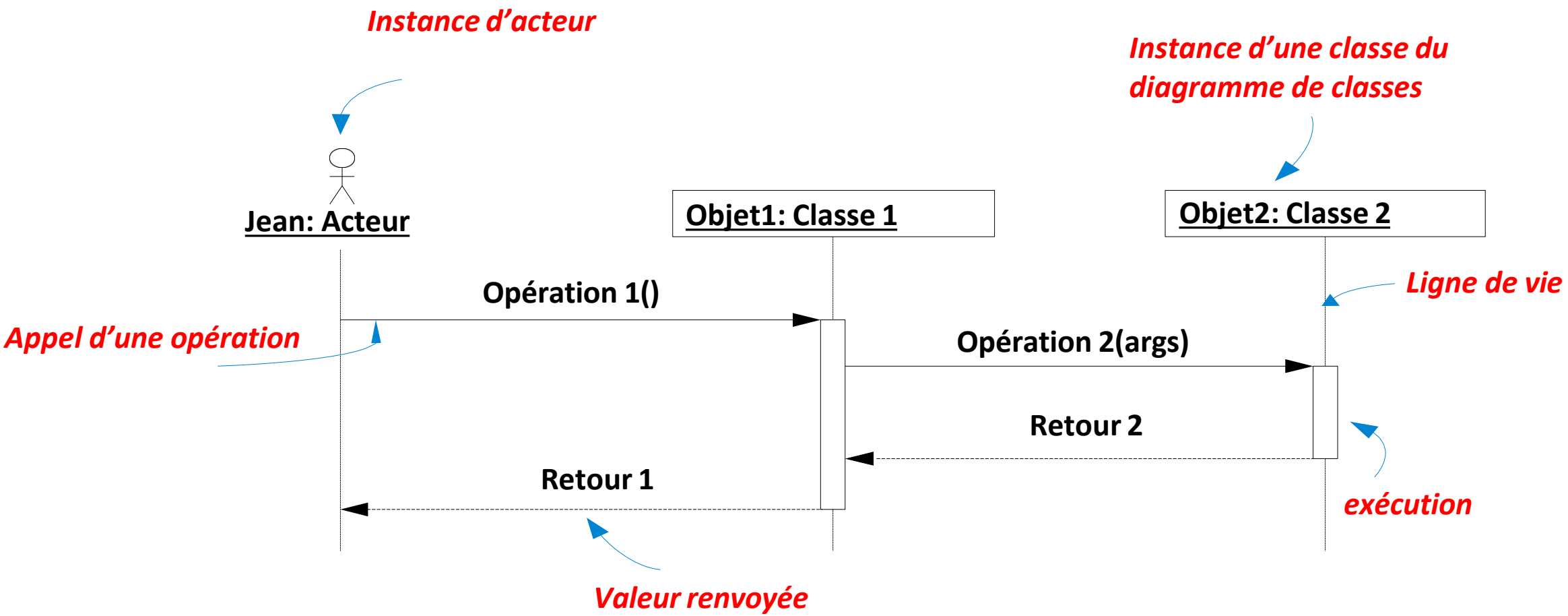
Représentation du diagramme de séquence

- Principe de base: Représentation graphique de la chronologie des échanges de messages avec le système ou au sein du système.

- Ligne de vie: « Vie » de chaque entité représentée verticalement.

- Echange de messages représentés horizontalement.

Eléments de base

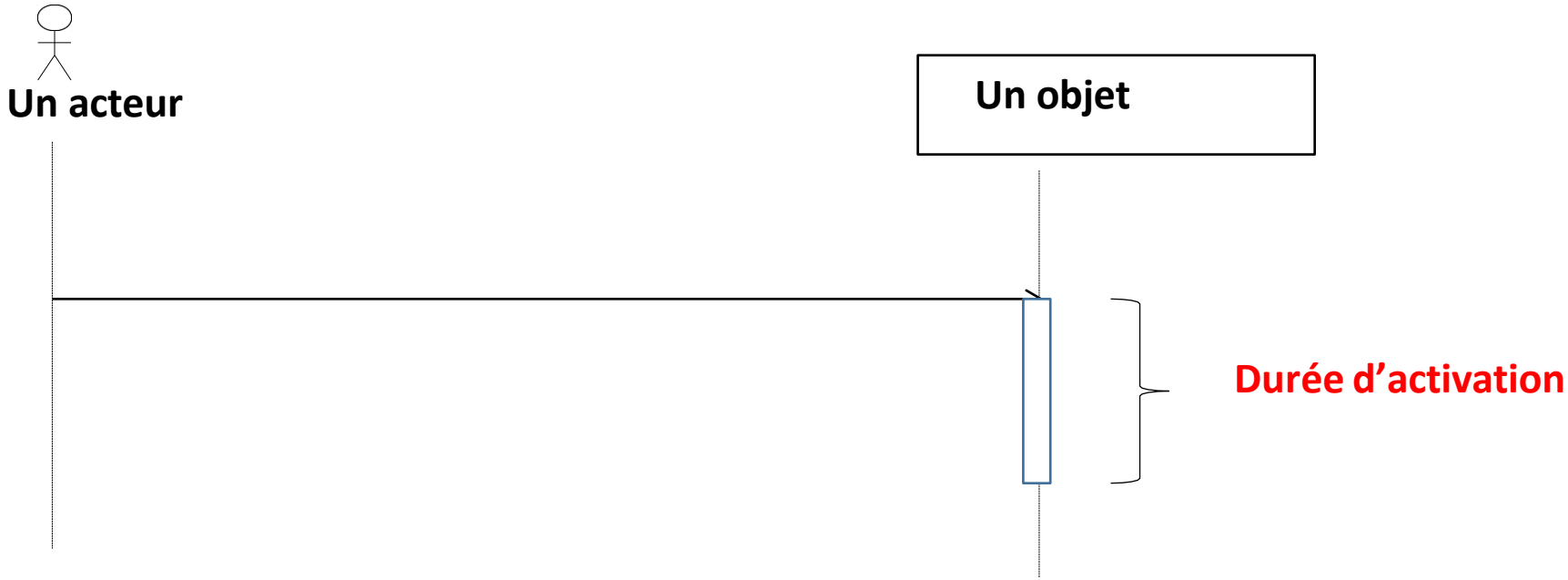


- Les interactions: L’interaction se traduit par l’envoi d’un message entre objets, le diagramme de séquence insiste sur la chronologie des objets en utilisant la ligne de vie des objets.

- Les activations: Les diagrammes de séquence permettent de représenter les périodes d’activité des objets.

- Une périodes d’activité: correspond au temps pendant lequel un objet effectue une action.

Formalisme: Les périodes d'activité se représentent par des bandes rectangulaires placées sur la ligne de vie des objets.



Les catégories de message

1. Message synchrone
2. Message asynchrone
3. Message de retour
4. Message réflexif

Chaque message possède:

- **Nom:** Qui est le nom de la méthode appelée ou du signal envoyé.

- **Une numérotation:** Devant le nom du message (séparée du nom par deux points « : »); la numérotation s'effectue séquentiellement à partir de 1.

- Les paramètres passés à la méthode ou au signal (entre parenthèse après le nom du message).

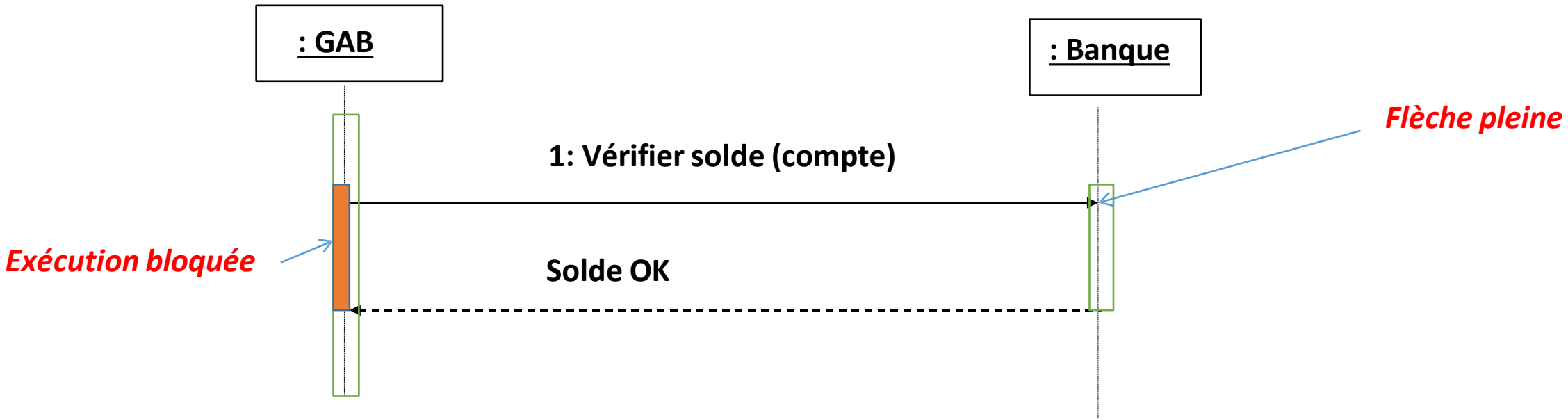
1- Message synchrone:

- Emetteur bloqué en attente du retour; Un objet poursuit donc son exécution une fois le comportement initié par le message terminé.

- La réception d'un message synchrone doit provoquer chez le destinataire le lancement d'une de ses méthodes qui souvent porte le même nom que le message. L'expéditeur du message reste bloqué pendant toute l'exécution de la méthode et attend la fin de celle-ci avant de pouvoir lancer un nouveau message (c'est le message le plus fréquemment utilisé).

- **Formalisme:** Des flèches pleines symbolisent tels messages.

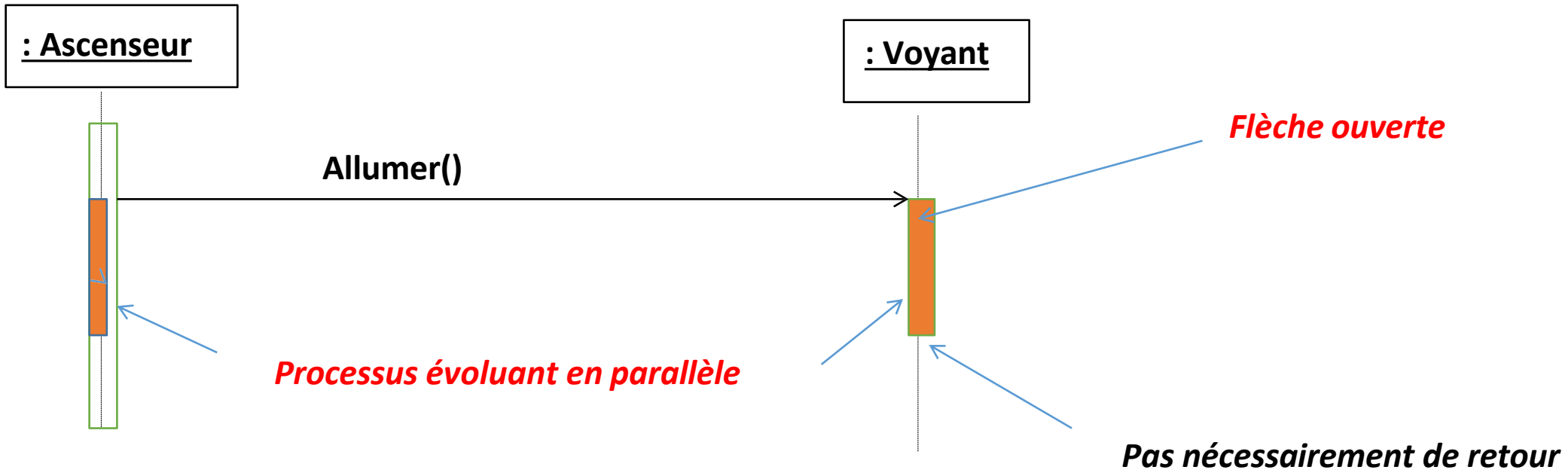
Exemple:


: GAB
: Banque
1: Vérifier solde (compte)
Solde OK
Flèche pleine
Exécution bloquée

2- Message asynchrone:

- Emetteur non bloqué, continue son exécution.
- Dans ce cas, l'expéditeur n'attend pas la fin de l'activation de la méthode invoquée chez le destinataire.
- **Formalisme:** Une flèches simple symbolise de tels messages.

Exemple:


: Ascenseur
: Voyant
Allumer()
Flèche ouverte
Processus évoluant en parallèle
Pas nécessairement de retour

3- Message de retour (réponse):

- Le retour de procédure est implicite à la fin d'une activation, Néanmoins, en cas d'envois de messages synchrones, il s'avère utile pour montrer la fin de l'exécution d'une sous-procédure et le renvoi éventuel de paramètres.

- **Formalisme:** Une flèches simple en pointillés.

Exemple:

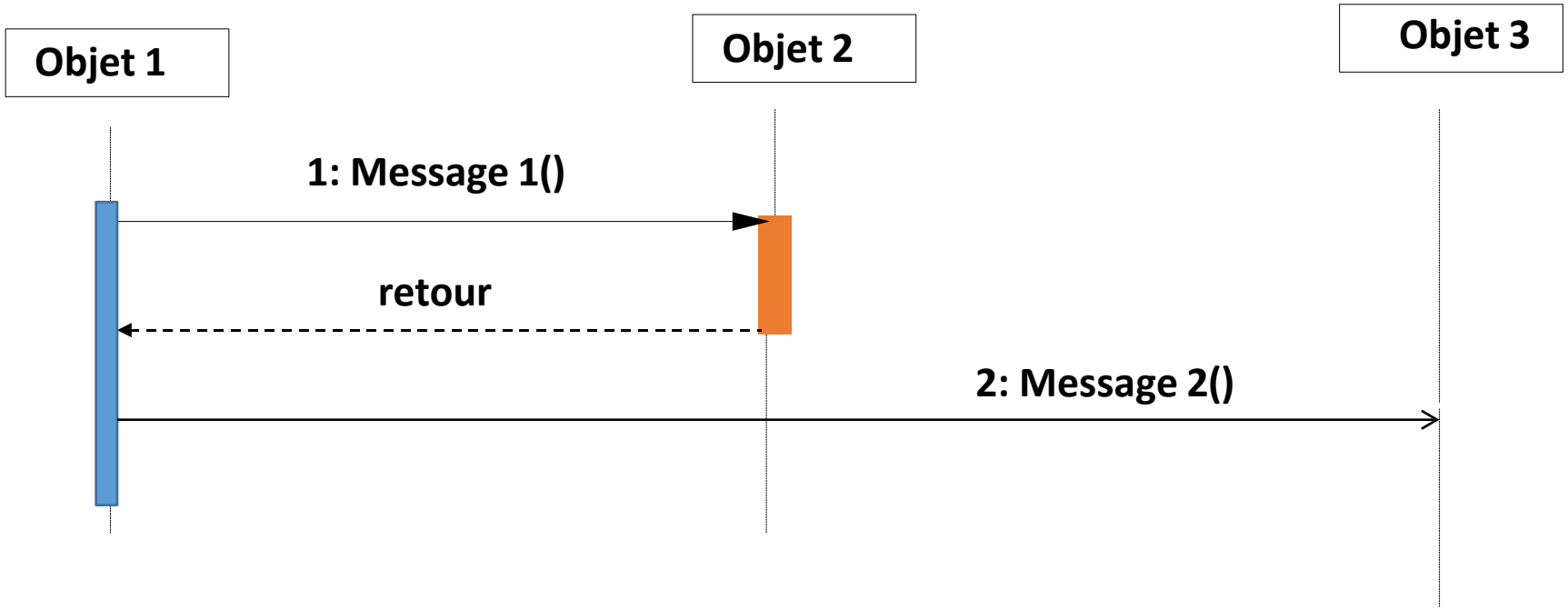
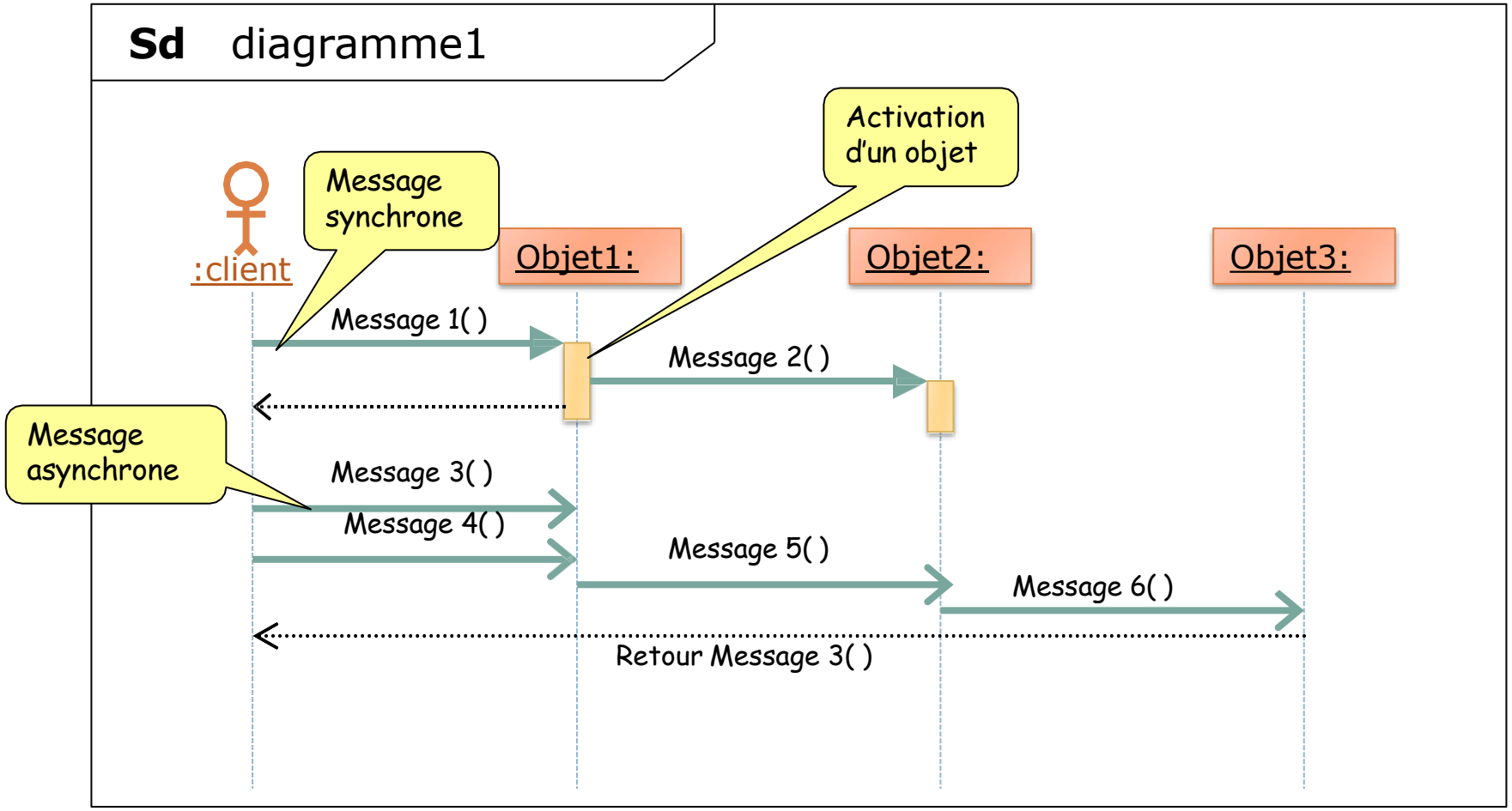


Diagramme de séquences : Types de messages

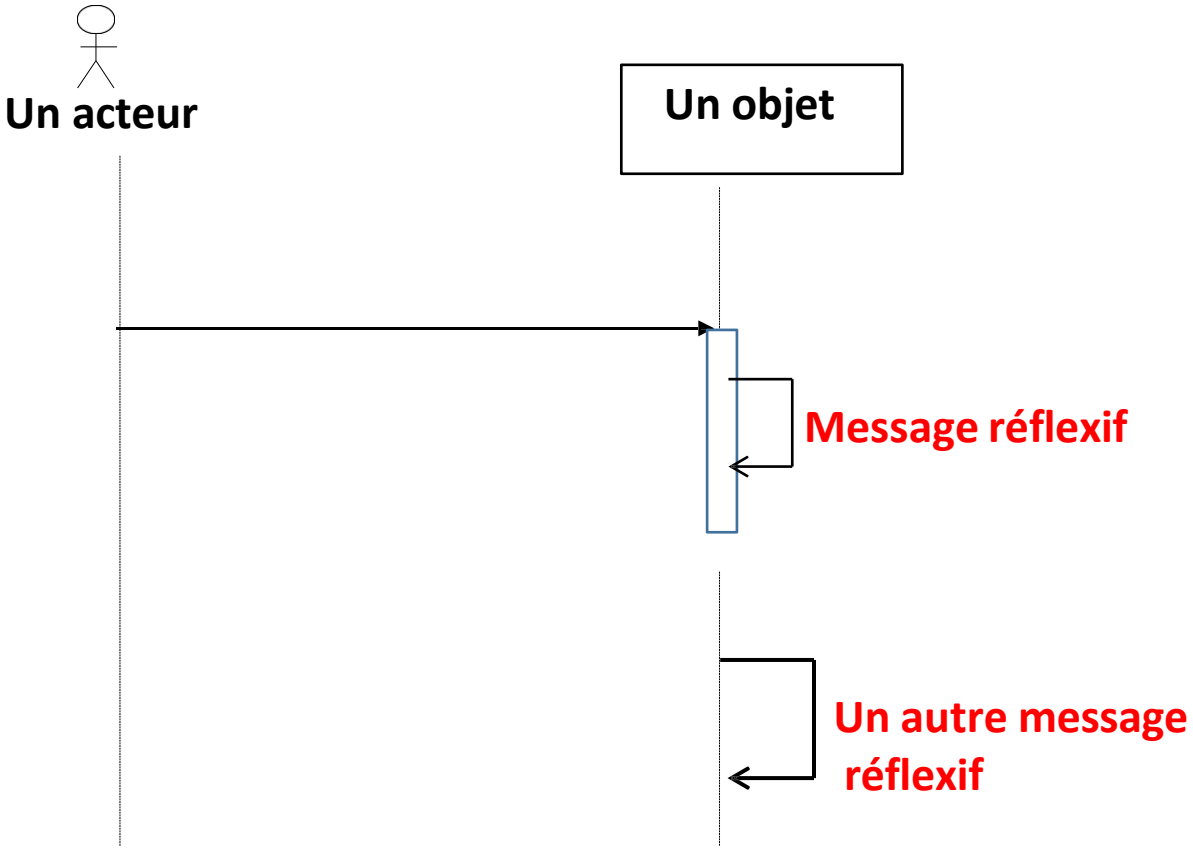


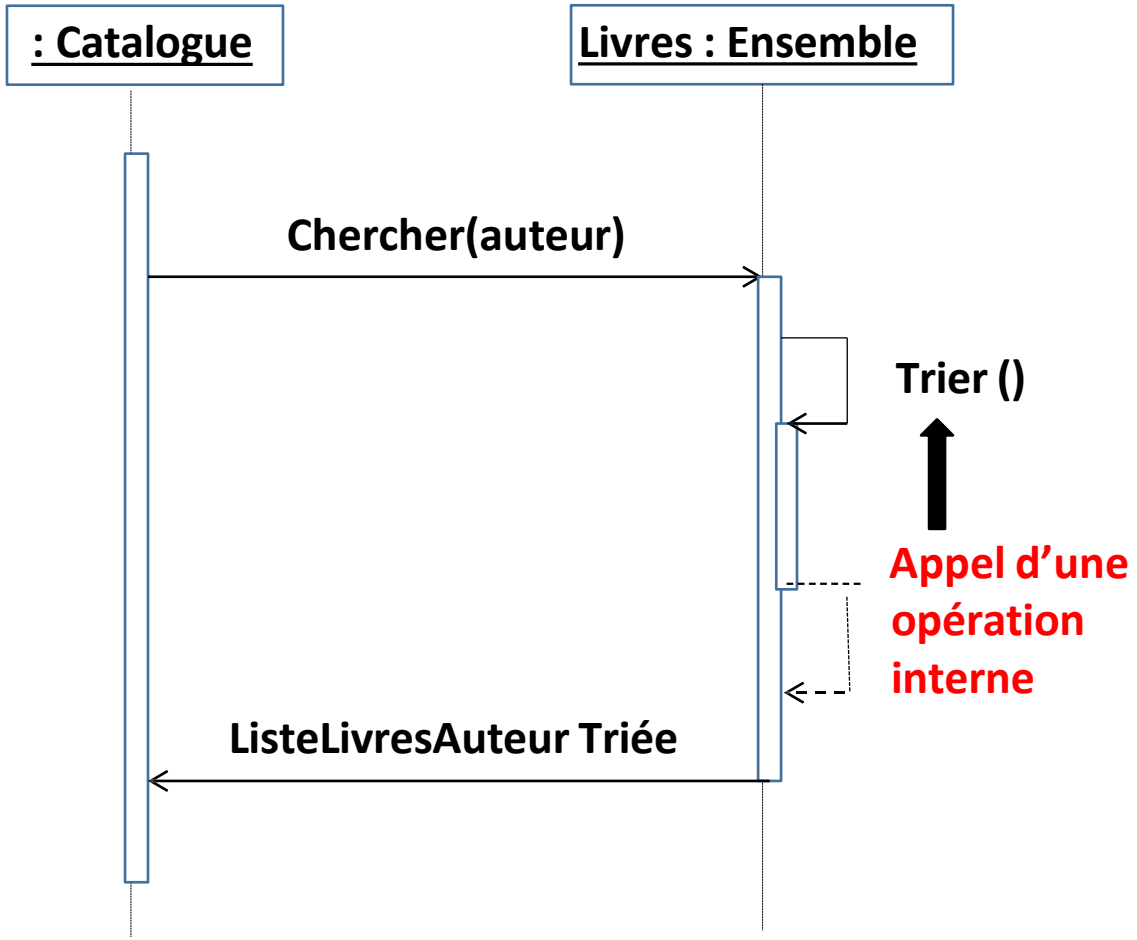
4- Message réflexif:

- Un objet peut s’envoyer un message.

- **Formalisme:** Cette situation se représente par une flèche qui revient en boucle sur la ligne de vie de l’objet.

Exemple:

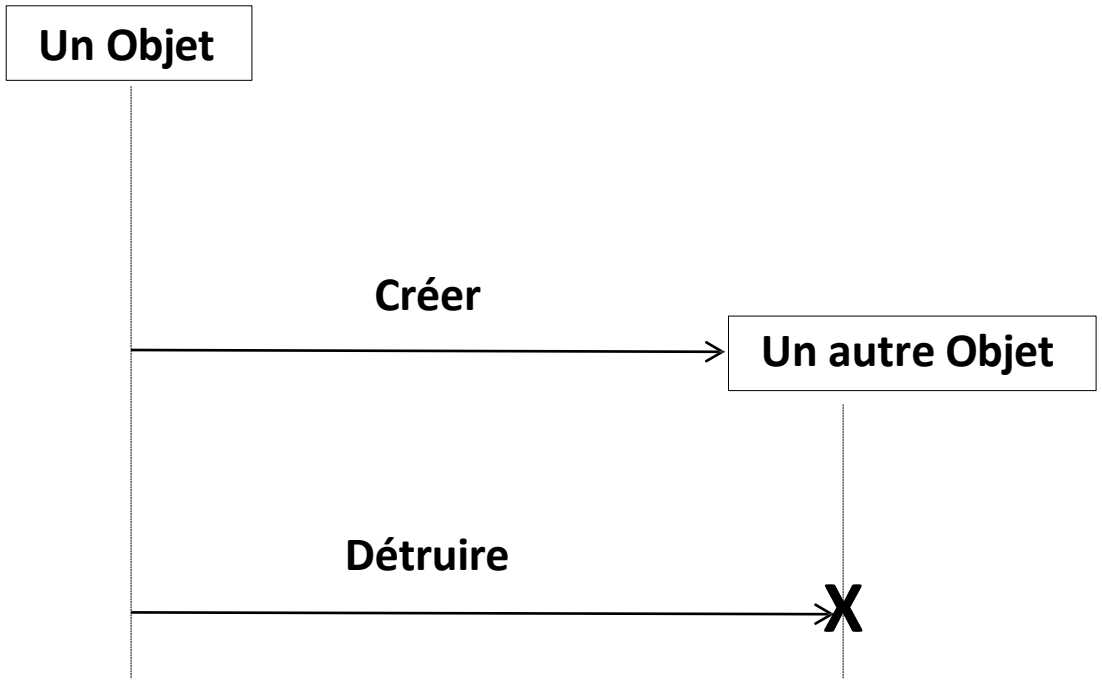

Un acteur
Un objet
Message réflexif
Un autre message réflexif

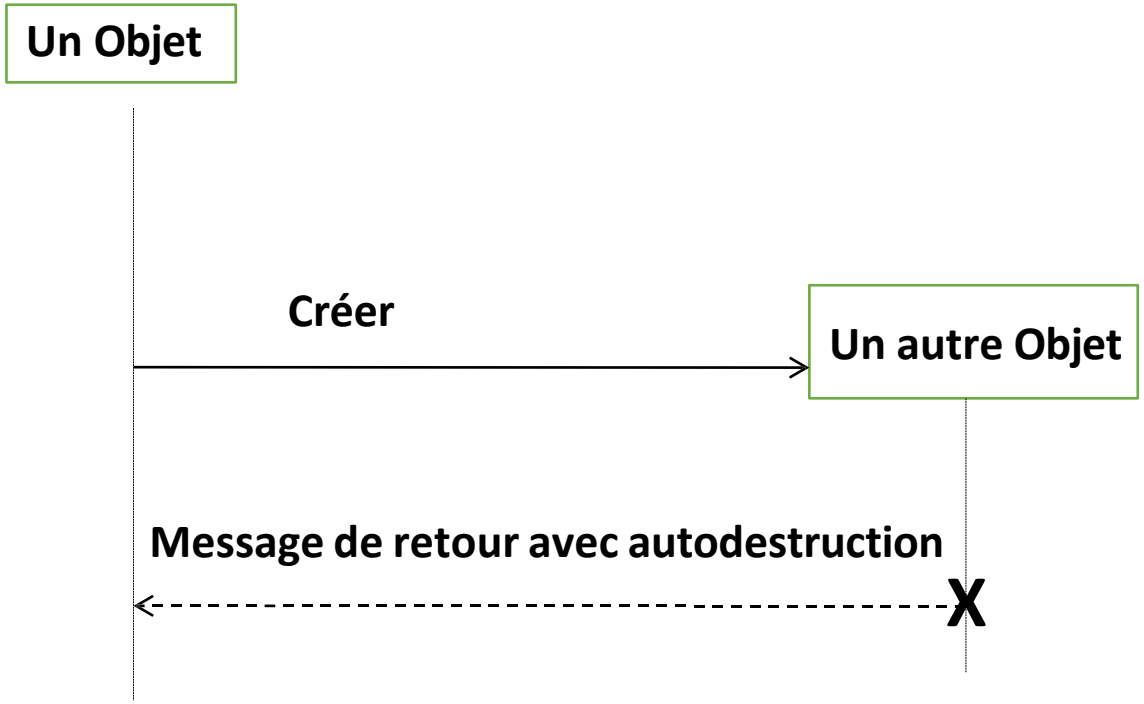

: Catalogue
Livres : Ensemble
Chercher(auteur)
Trier ()
Appel d'une opération interne
ListeLivresAuteur Triée

Création et destruction d'objet

- La ligne de vie des objets est représentée par une ligne verticale en trais pointillés, placée sous le symbole de l'objet concerné. Cette ligne de vie précise l'existence de l'objet concerné durant un certain laps de temps.
- En général, une ligne de vie est représentée sous toute la hauteur du diagramme de séquence, Par contre, elle peut débuter et s'interrompre à l'intérieur du diagramme.
- Formalisme: La création se représente en faisant pointer le message de création sur le rectangle qui symbolise l'objet crée. La destruction est indiquée par la fin de la ligne de vie et par une croix (X), soit à la hauteur du message qui cause la destruction, soit après le dernier message envoyé par un objet qui se suicide.

Exemple 1:


Un Objet
Créer
Un autre Objet
Détruire
X


Un Objet
Créer
Un autre Objet
Message de retour avec autodestruction
X

Exemple 2:

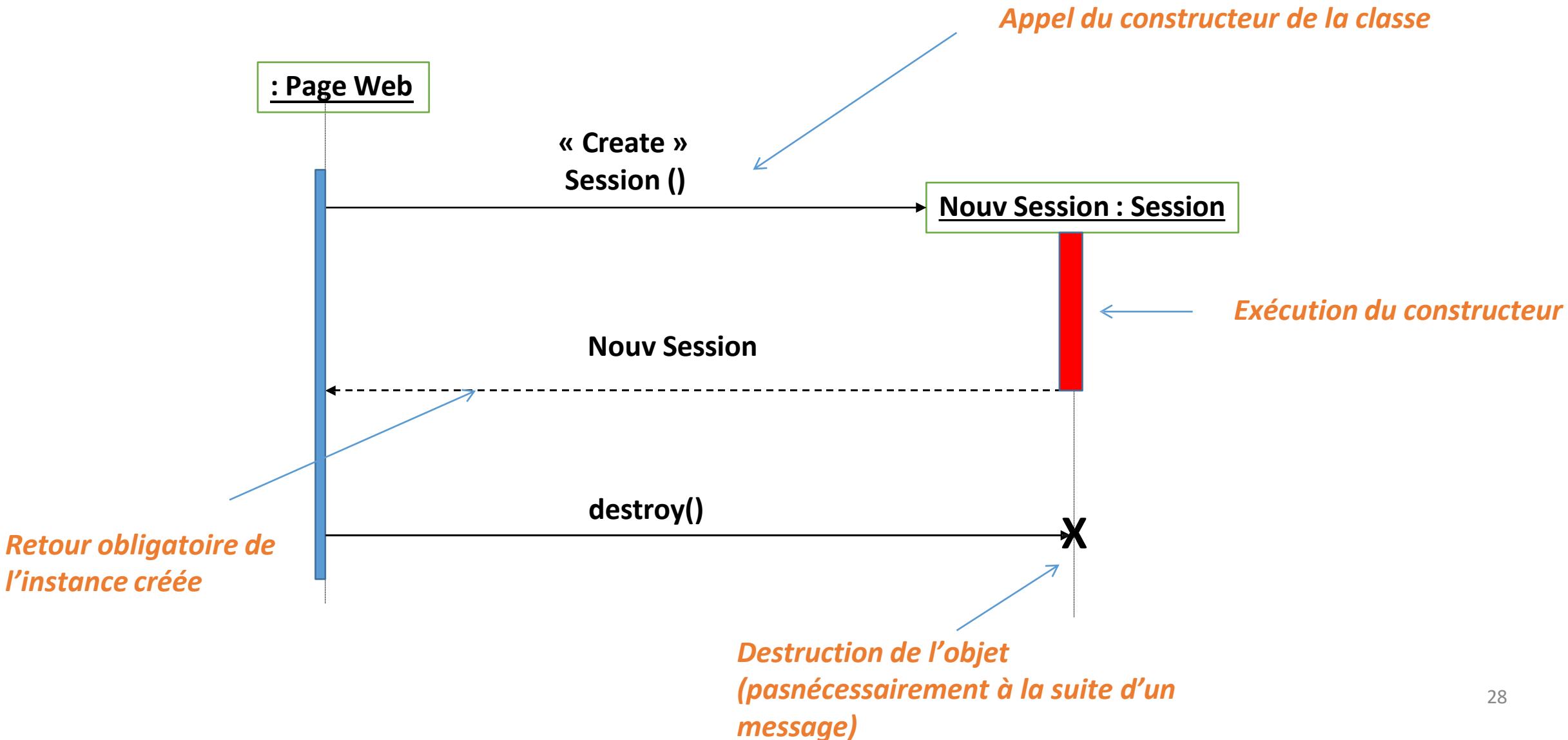

Appel du constructeur de la classe
: Page Web
« Create »
Session ()
Nouv Session : Session
Exécution du constructeur
Nouv Session
destroy()
Retour obligatoire de l'instance créée
Destruction de l'objet (pasnécessairement à la suite d'un message)

Diagramme de séquences : Types de messages

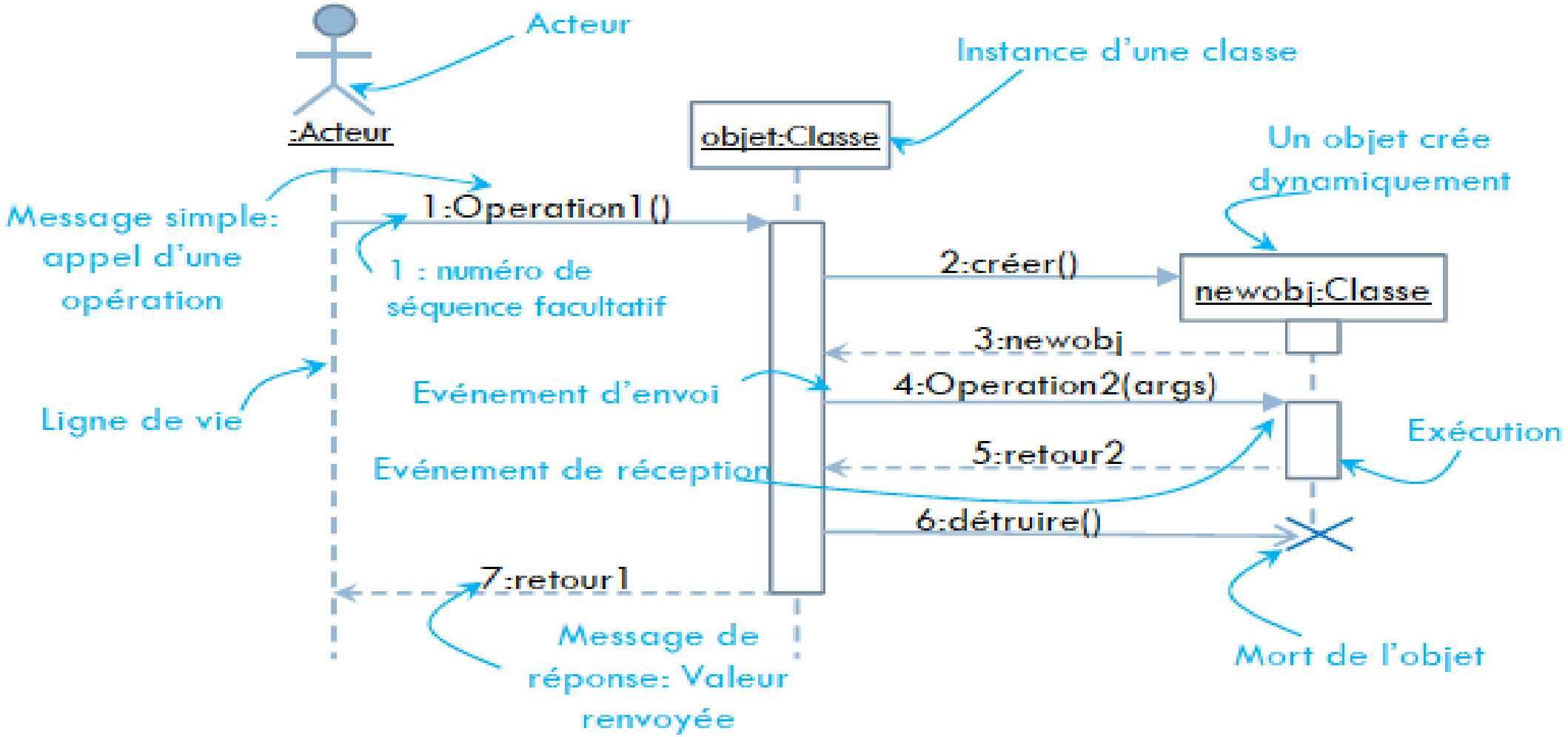


Diagramme de séquences :
Exemple Complet: Scénario Retrait d'argent

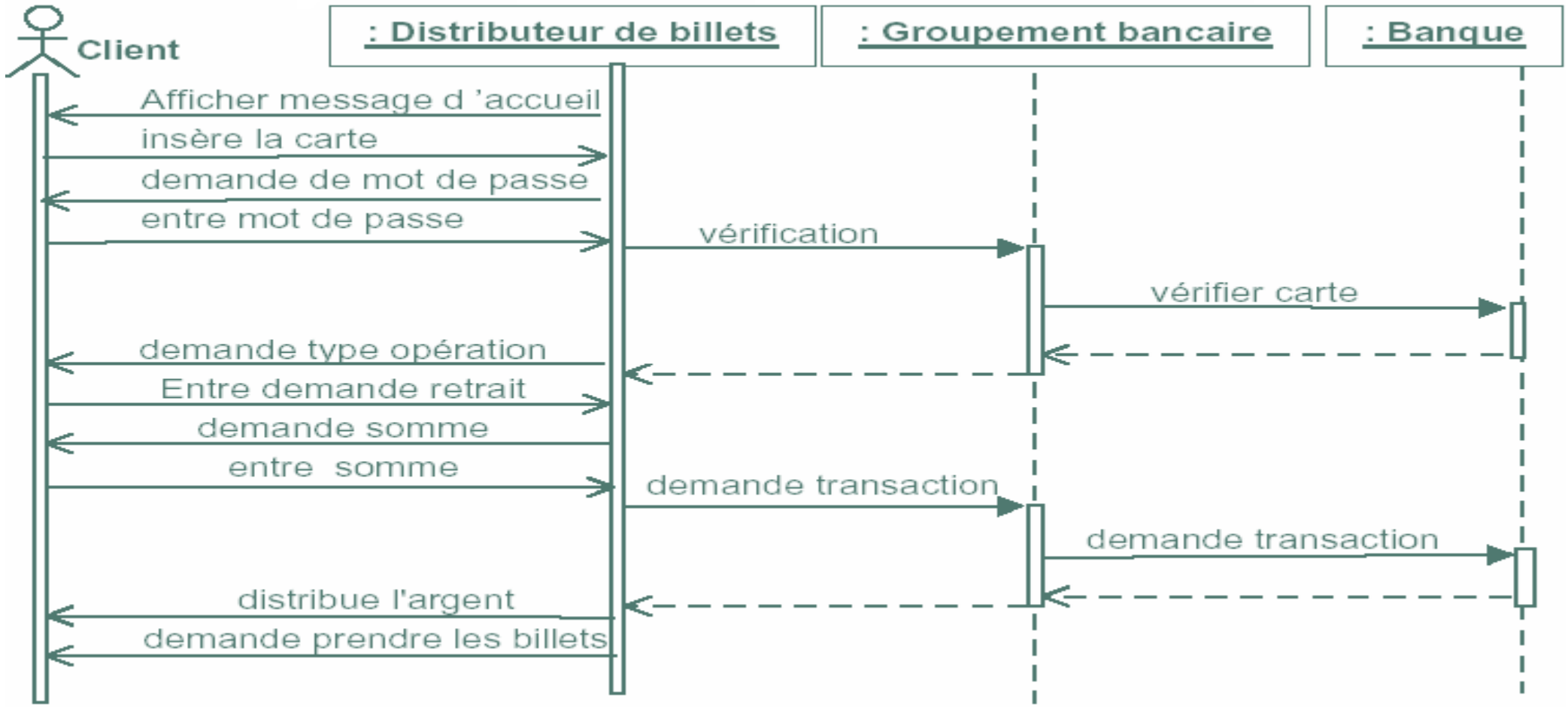
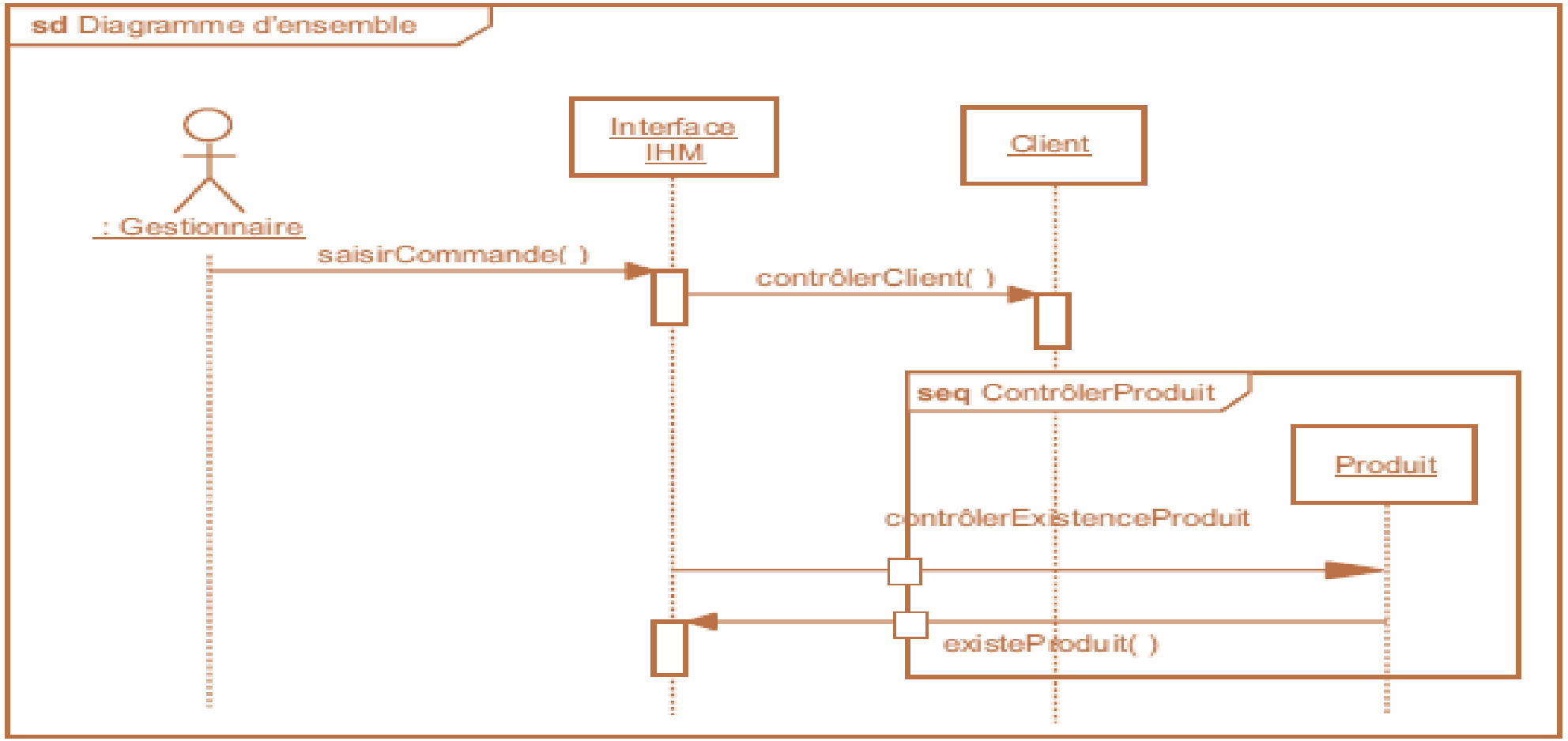


Diagramme de séquence: Fragment d'interaction

- Un **Fragment d'interaction** se représente globalement comme un diagramme de séquence dans un rectangle avec indication dans le coin à gauche du nom du fragment.
- Un Fragment d'interaction permet d'identifier une sous-partie d'une interaction afin que celle-ci soit référencées par d'autres interactions ou de lui spécifier des conditions particulières d'exécution (boucle, alternative, référence…)
- Un port d'entrée et un port de sortie peuvent être indiqués pour connaître la manière dont ce fragment peut être relié au reste du diagramme. Dans le cas où aucun port n'est indiqué c'est l'ensemble du fragment qui est appelé pour exécution.

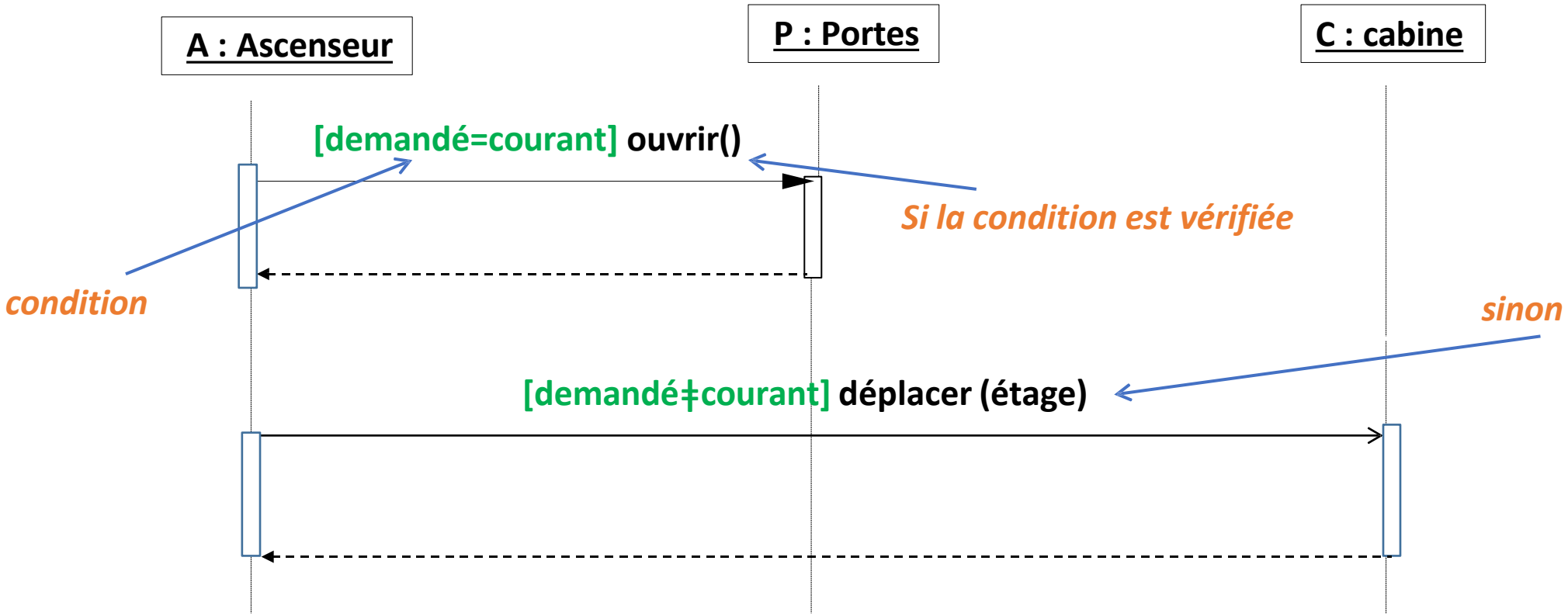

sd Diagramme d'ensemble
: Gestionnaire
Interface IHM
Client
saisirCommande( )
contrôlerClient( )
seq ContrôlerProduit
Produit
contrôlerExistenceProduit
existeProduit( )

- **Un fragment d'interaction dit combiné** correspond à un ensemble d'interactions auquel on applique un opérateur.
- Un fragment combiné se représente globalement comme un diagramme de séquence avec indication dans le coin à gauche du nom de l'opérateur.
- **13 opérateurs** ont été définis dans UML : alt, opt, loop, par, strict/weak, break, ignore/consider, critical, negative, assertion et ref.

L'opérateur alt: Alternative

Condition à l'envoi d'un message

Notation: Deux diagrammes

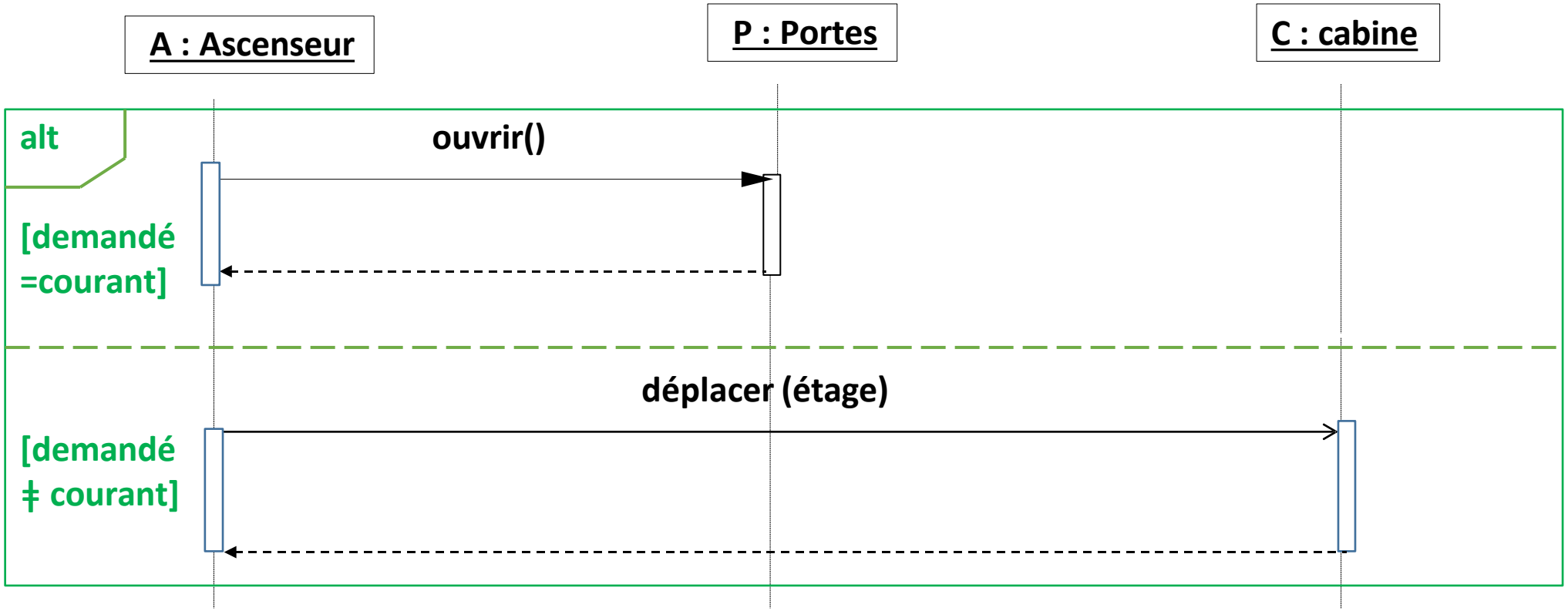


Alt

Condition à l'envoi d'un message

Notation: Deux diagrammes

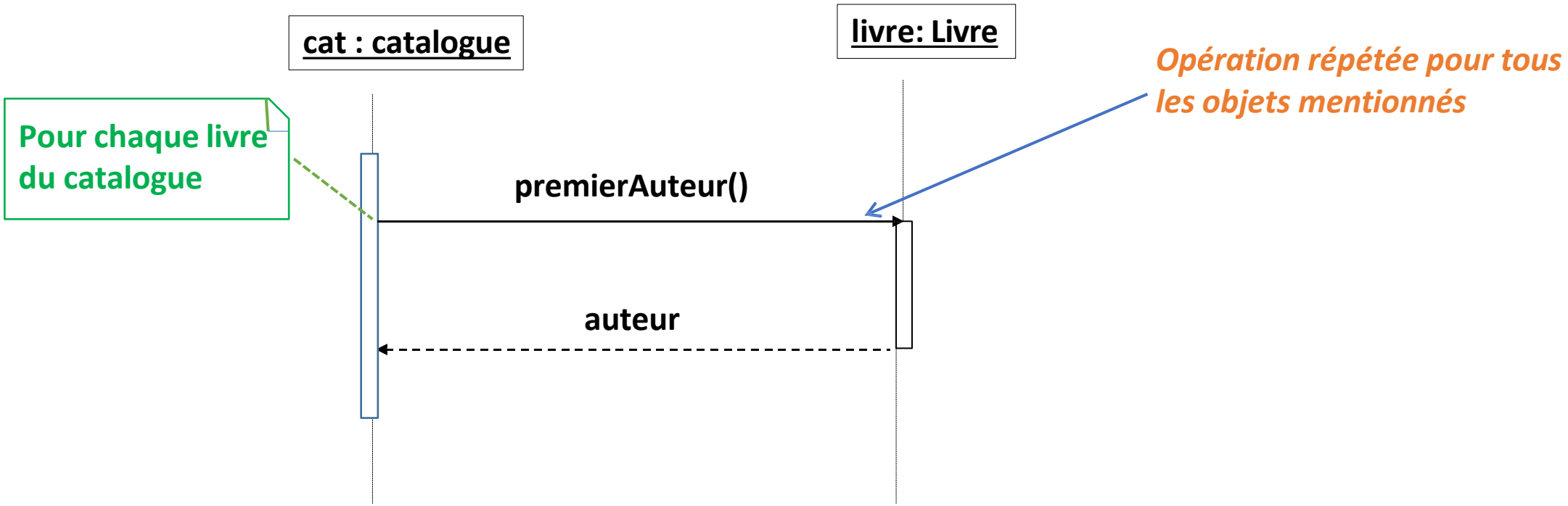
Bloc d'alternative **alt**



L'opérateur Loop: Boucle

Répéter un enchaînement de messages

Notation: Note

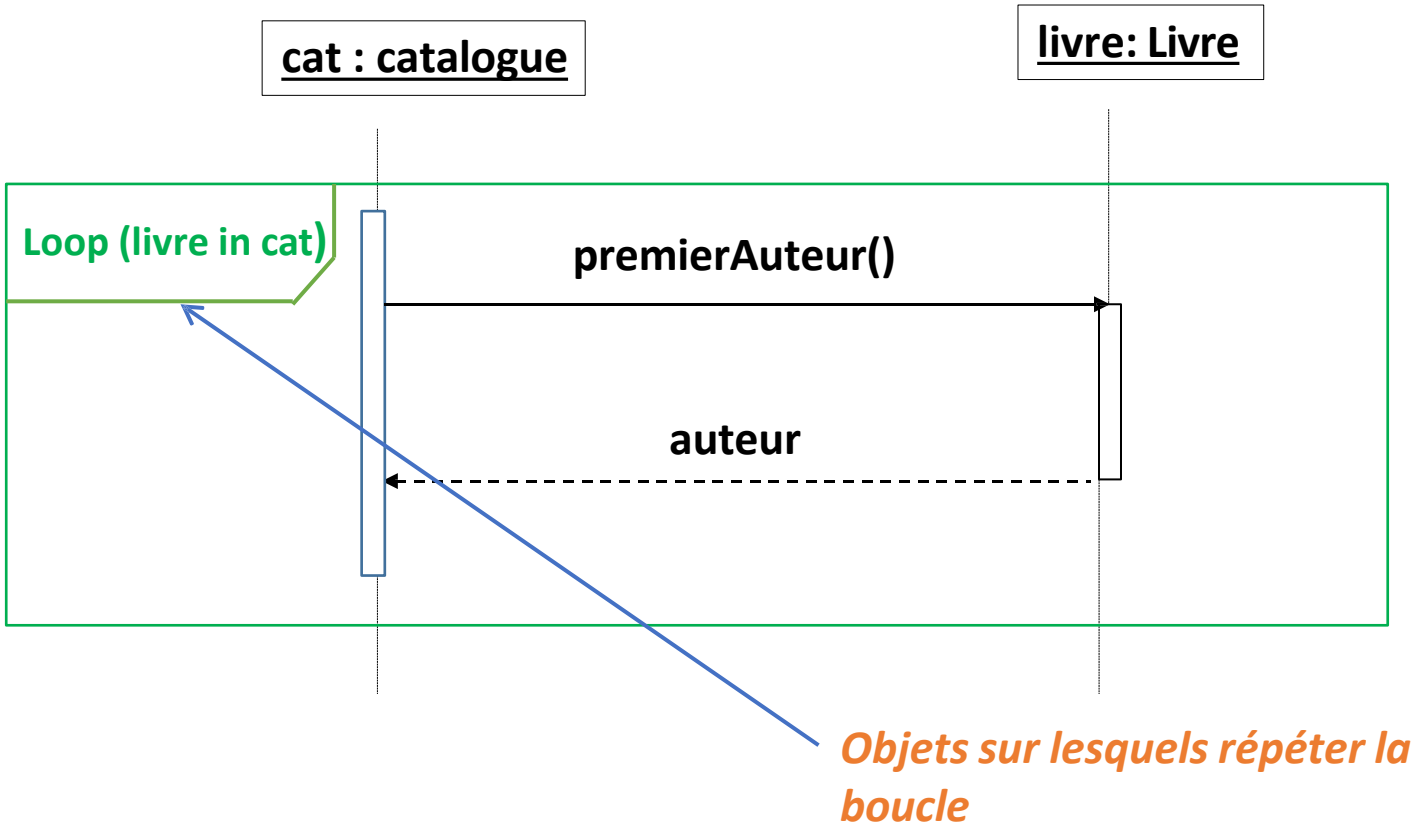


Loop

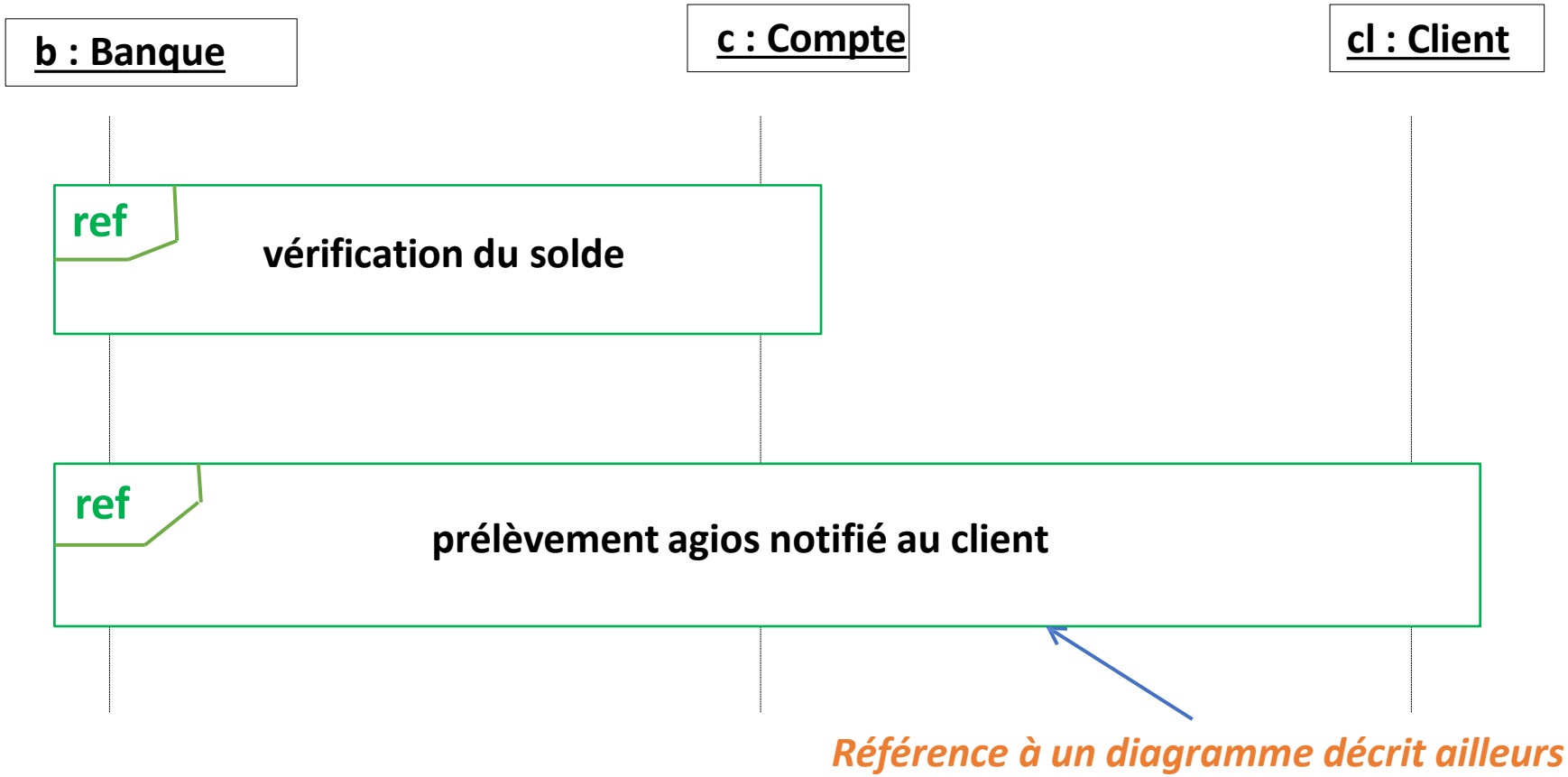
Répéter un enchaînement de messages

Notation: Note

Bloc de boucle **loop**

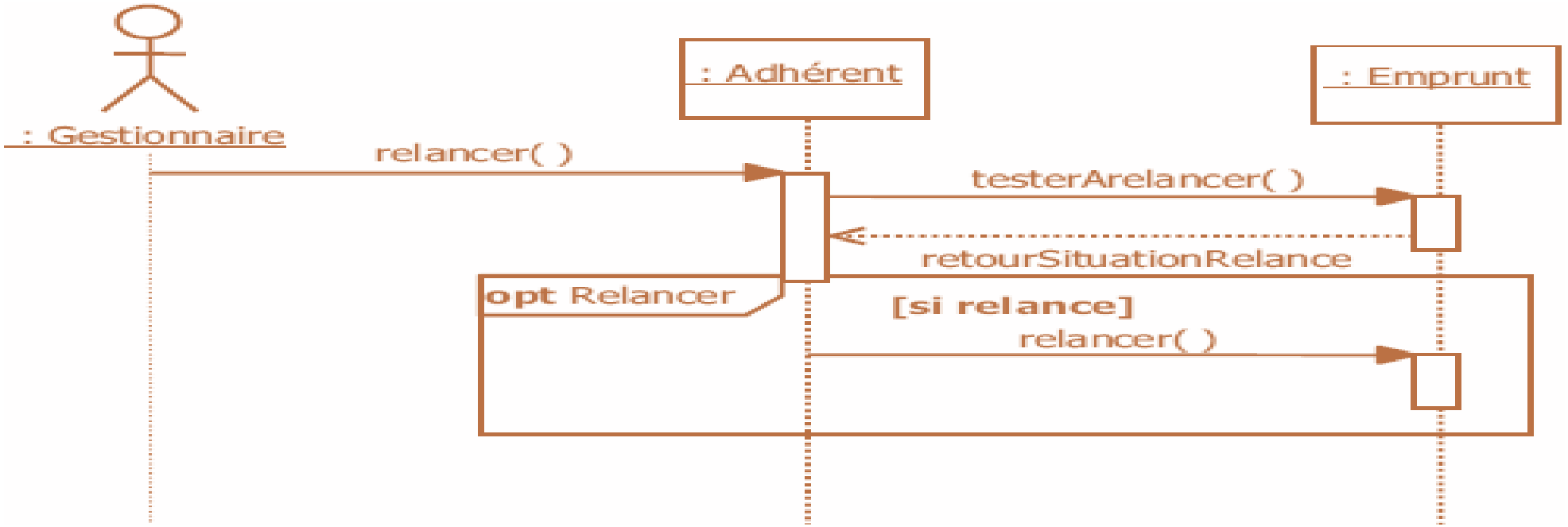


L’opérateur Ref: Référence à un autre diagramme



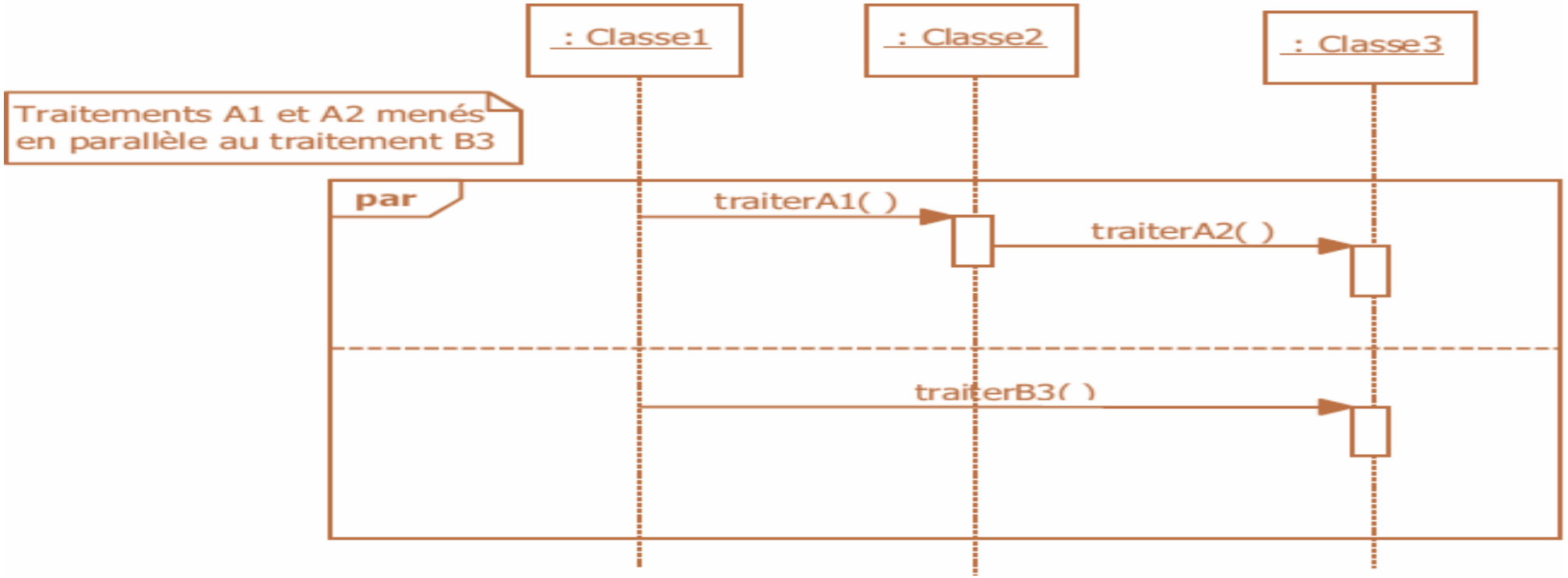
L’opérateur Opt

- Désigne un fragment combiné optionnel comme son nom l’indique: c’est-à-dire qu’il représente un comportement qui peut se produire ou pas. Un fragment optionnel est équivalent à un fragment ‘alt’ qui ne posséderait pas d’opérande else « Si…Alors ».



L’opérateur Parrallèle (Par)

- **Opérateur Par (Parallel):** Il est utilisé pour représenter des interactions ayant lieu en parallèle. Les interactions des différents opérandes peuvent donc se mélanger, s’intercaler, dans la mesure où l’ordre imposé dans chaque opérande est respecté.

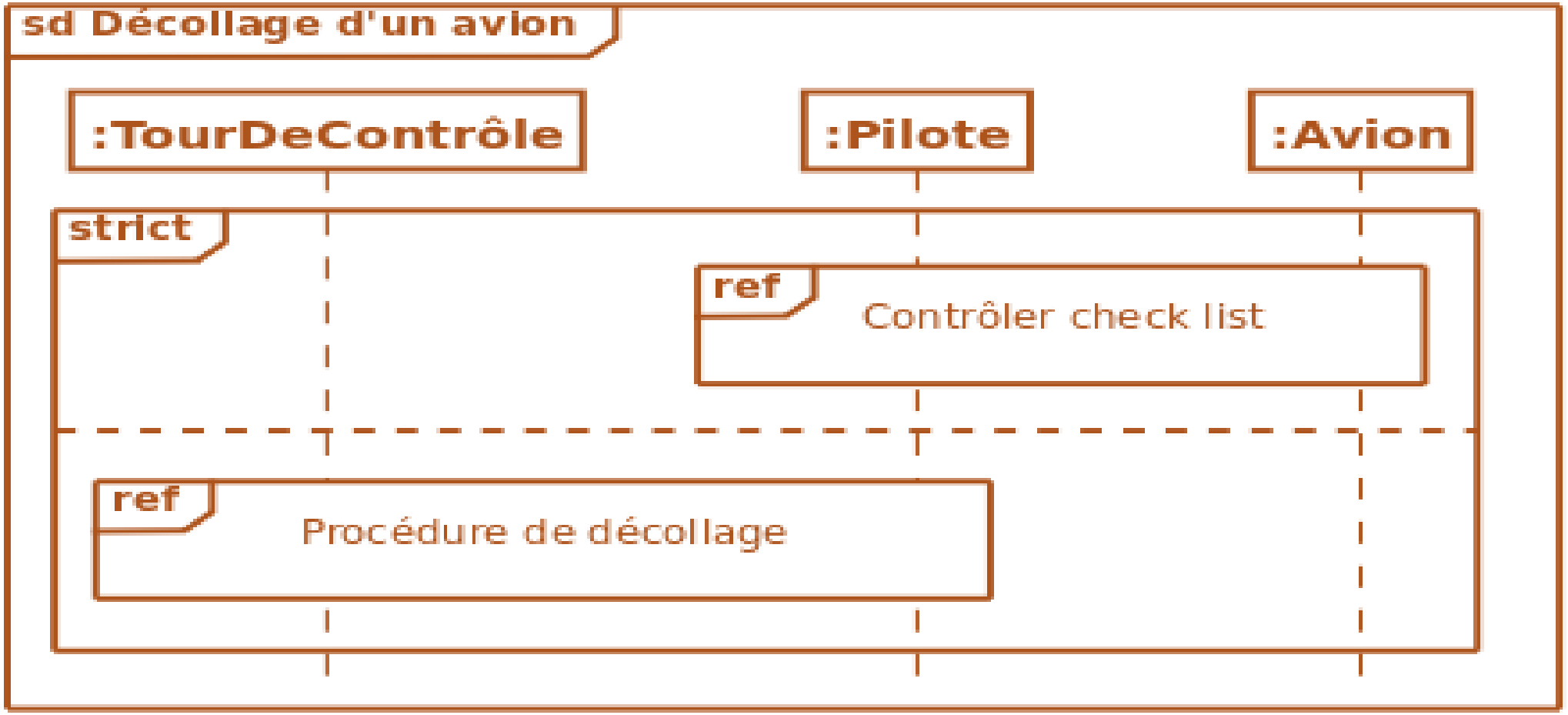


Les opérateurs strict et weak sequencing

- Les opérateurs **srict** et **weak** permettent de représenter une série d'interactions dont certaines s'opèrent sur des objets indépendants :

  - L'opérateur ***strict*** est utilisé quand l'ordre d'exécution des opérations doit être strictement respecté.

  - L'opérateur ***weak*** est utilisé quand l'ordre d'exécution des opérations n'a pas d'importance.



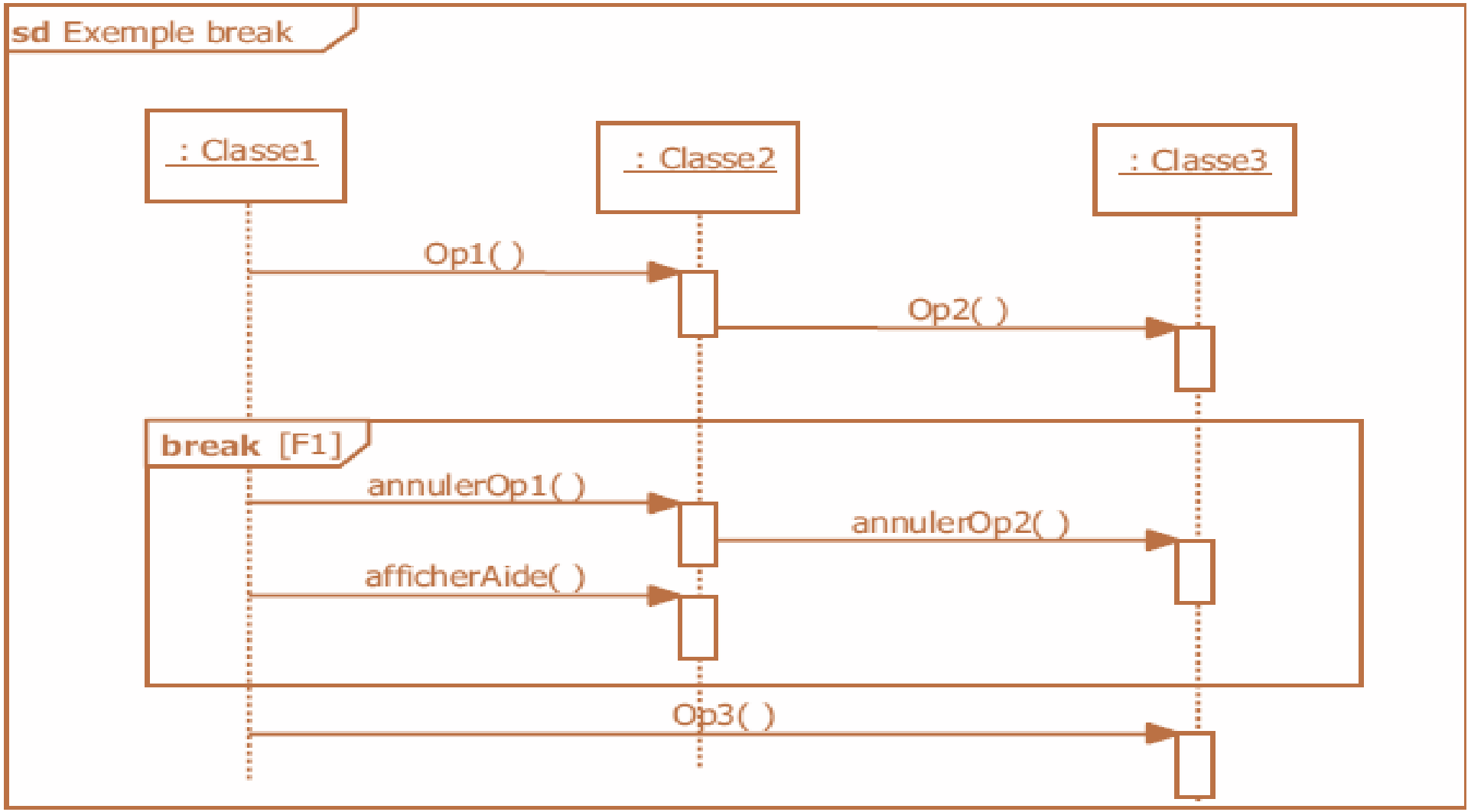
Procédures de décollage d'un avion dans l'ordre.

L’opérateur Break

- **Opérateur Break:** Il est utilisé dans les fragments combinés qui représentent des scénarios d’exception en quelque sorte. Les interactions de ce fragment seront exécutées à la place des interactions décrites en dessous. Il y a donc une notion d’interruption du flot normal des interactions.

- L’opérateur **break** permet de représenter une situation exceptionnelle correspondante à un scénario de rupture par rapport au scénario général.

- Le scénario de rupture s’exécute si la condition de garde est satisfaite.


sd Exemple break
: Classe1
: Classe2
: Classe3
Op1( )
Op2( )
break [F1]
annulerOp1( )
annulerOp2( )
afficherAide( )
Op3( )

Stéréotypes de Jacobson

- A l'intérieur d'un système, il existe très souvent des classes qui possèdent un rôle bien particulier qui serait intéressant de visualiser d'une façon non équivoque dans le diagramme de séquence.
- C'est le cas notamment:
  - Pour les classes qui représentent des composants de l'IHM.
  - Pour la classe qui contrôle globalement le système avec la prise en compte de la gestion événementielle.
  - Pour les classes qui implémentent la persistance des attributs (qui sont associées à une base de données).

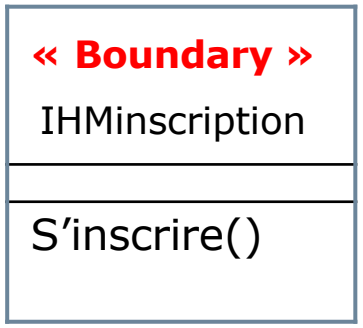
- Jacobson distingue les trois stéréotypes suivants:

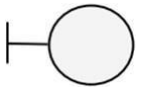
1- Présentation → «Boundary »

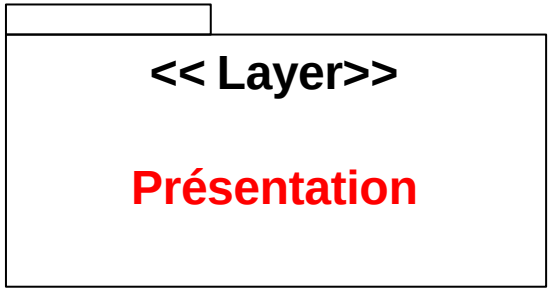
2- Application → «Control »

3- Métier → «Entity »

Stéréotypes de Jacobson ⇒ Architecture à 3 couches



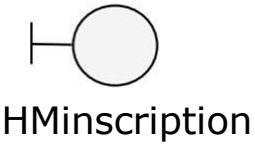


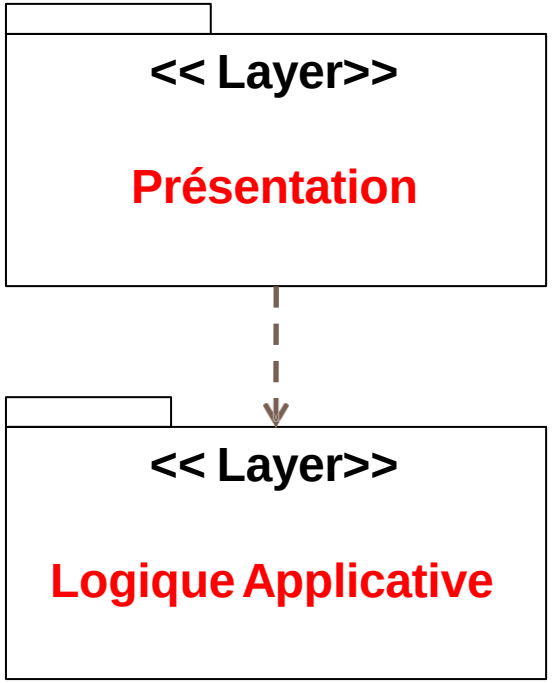


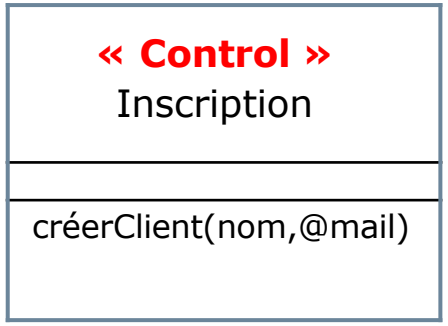
<<boundary>> : classes qui servent à modéliser les interactions entre le système et ses acteurs ;

Stéréotypes de Jacobson ⇒ Architecture à 3 couches









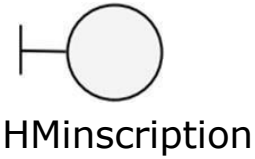


<<control>> : classes utilisées pour représenter la coordination, l'enchaînement et le contrôle d'autres objets – elles sont en général reliées à un cas d'utilisation particulier ;

Stéréotypes de Jacobson

➡ Architecture à 3 couches

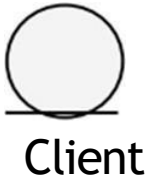
« Boundary »
IHMinscription

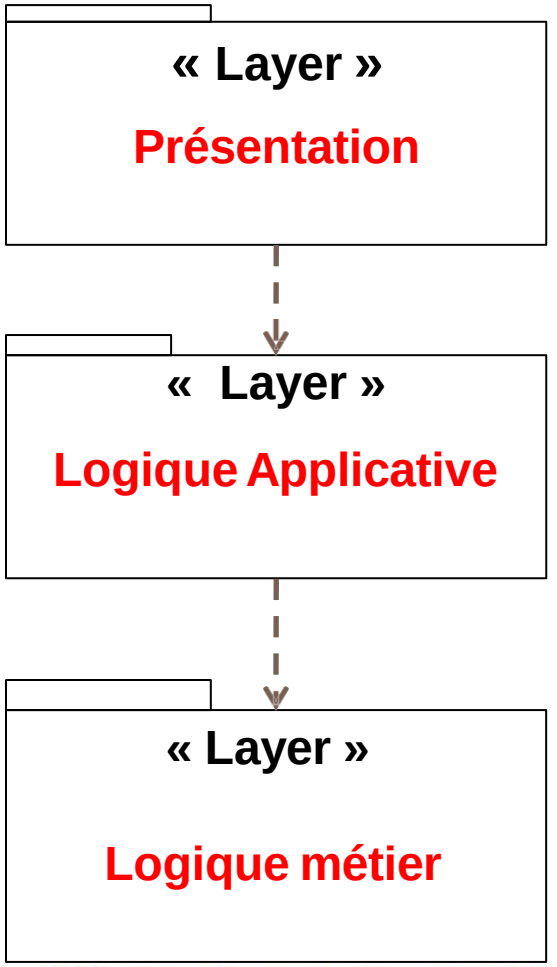


« control »
Inscription



« Entity»
Client
create(nom,@mail)

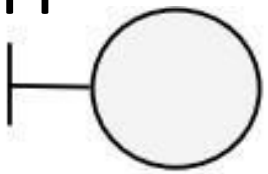




<<entity>> : classes qui servent à modéliser des informations durables et souvent persistantes

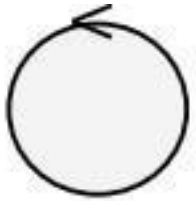
Stéréotypes de Jacobson

➡ Architecture à 3 couches



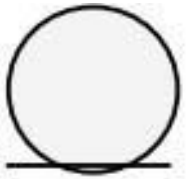
: <Boundary Name>

<<boundary>> : classes qui servent à modéliser les interactions entre le système et ses acteurs ;



: <Control Name>

<<control>> : classes utilisées pour représenter la coordination, l'enchaînement et le contrôle d'autres objets – elles sont en général reliées à un cas d'utilisation particulier ;



: <Entity Name>

<<entity>> : classes qui servent à modéliser des informations durables et souvent persistantes

Stéréotypes de Jacobson ⇨ Architecture à 3 couches

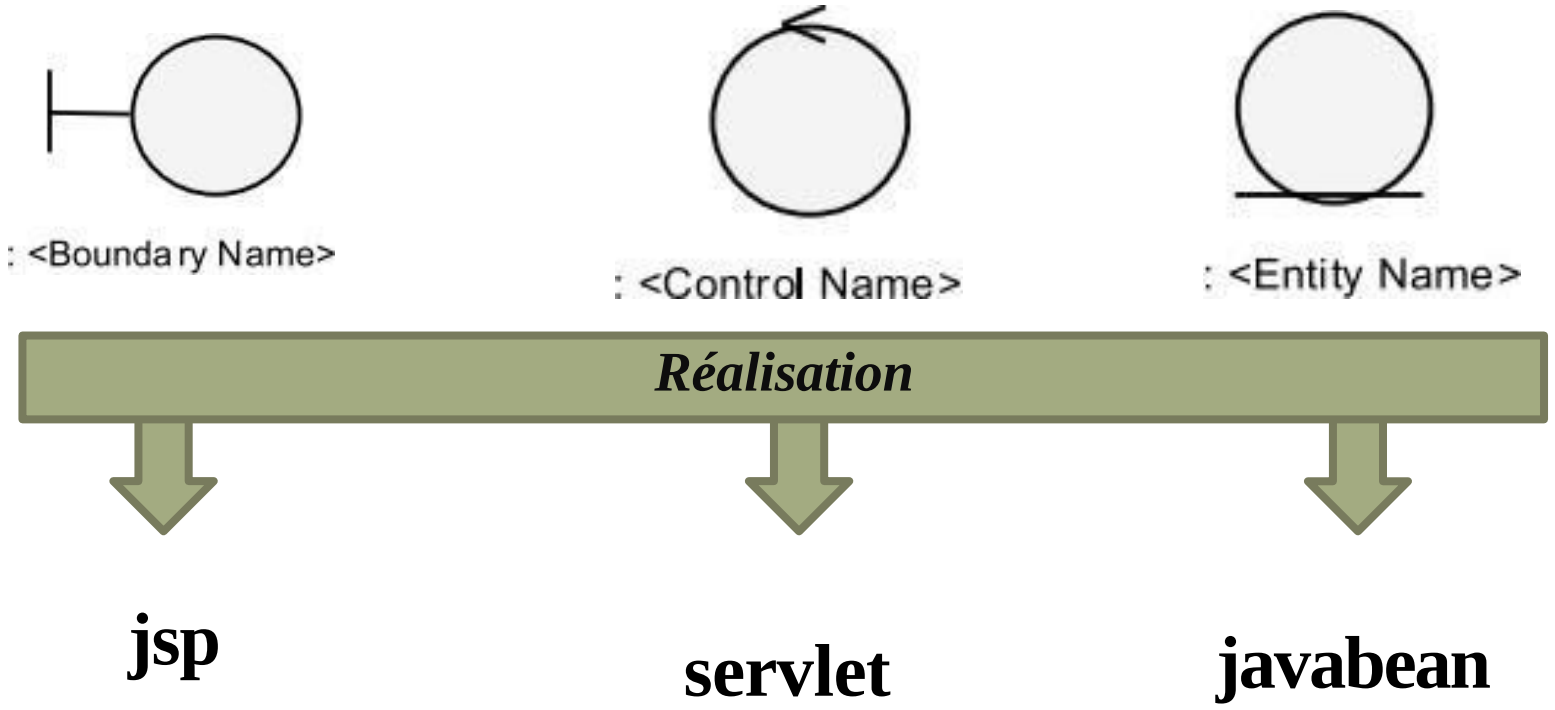


Diagramme de séquence objet (enrichit des Stéréotypes de Jacobson)…

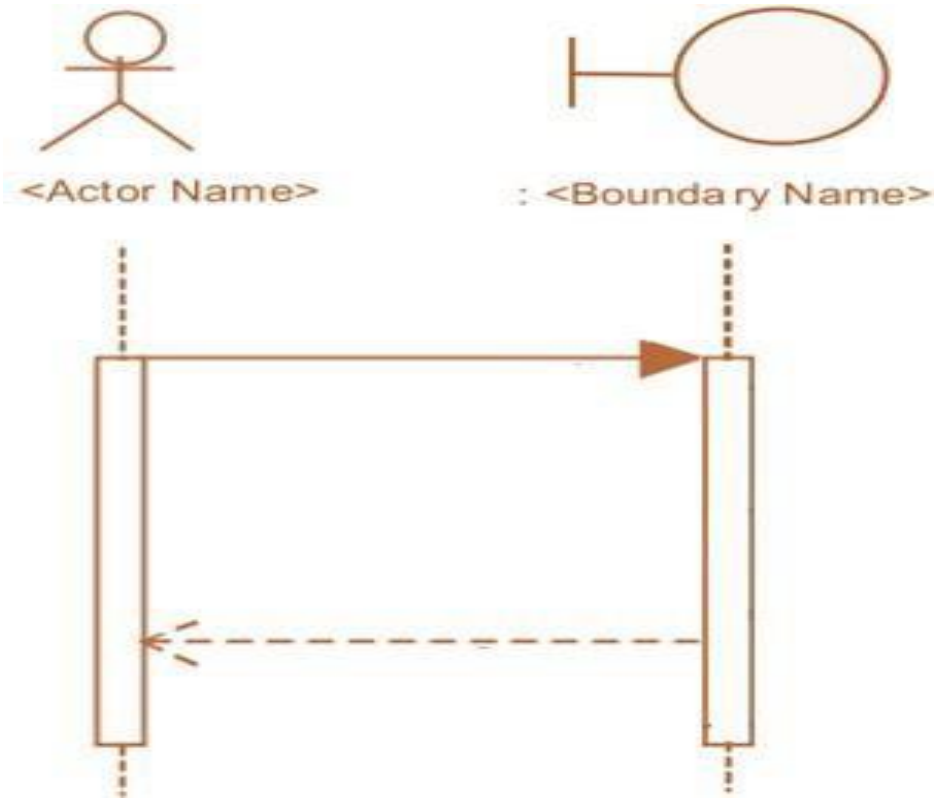


Diagramme de séquence objet…

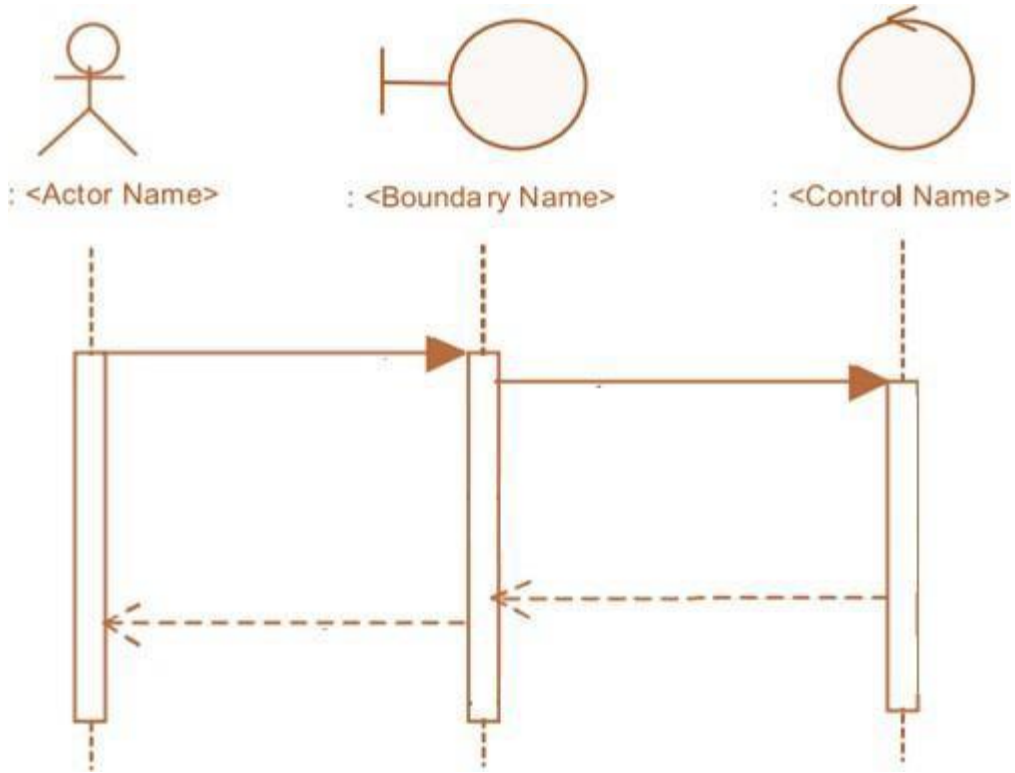


Diagramme de séquence objet…

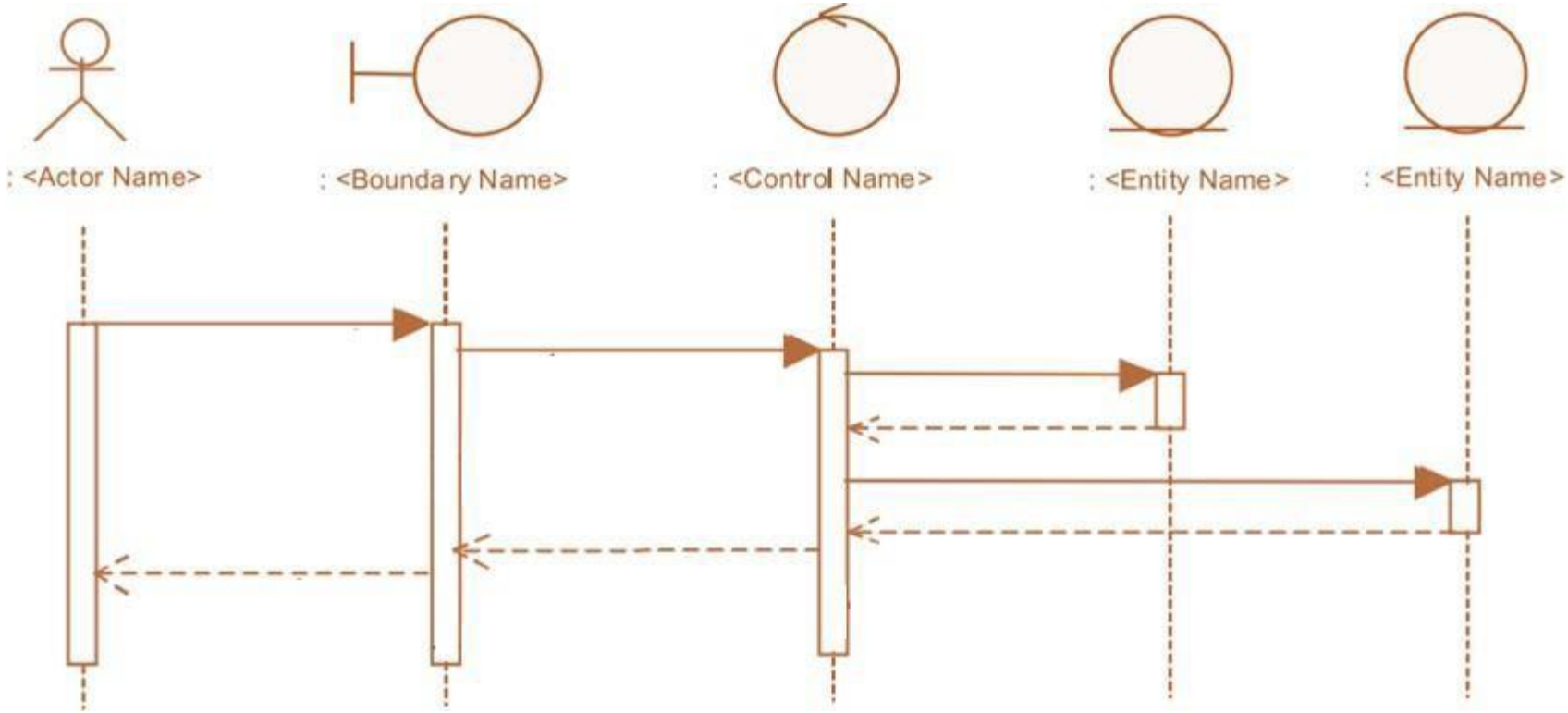


Diagramme de séquence objet…

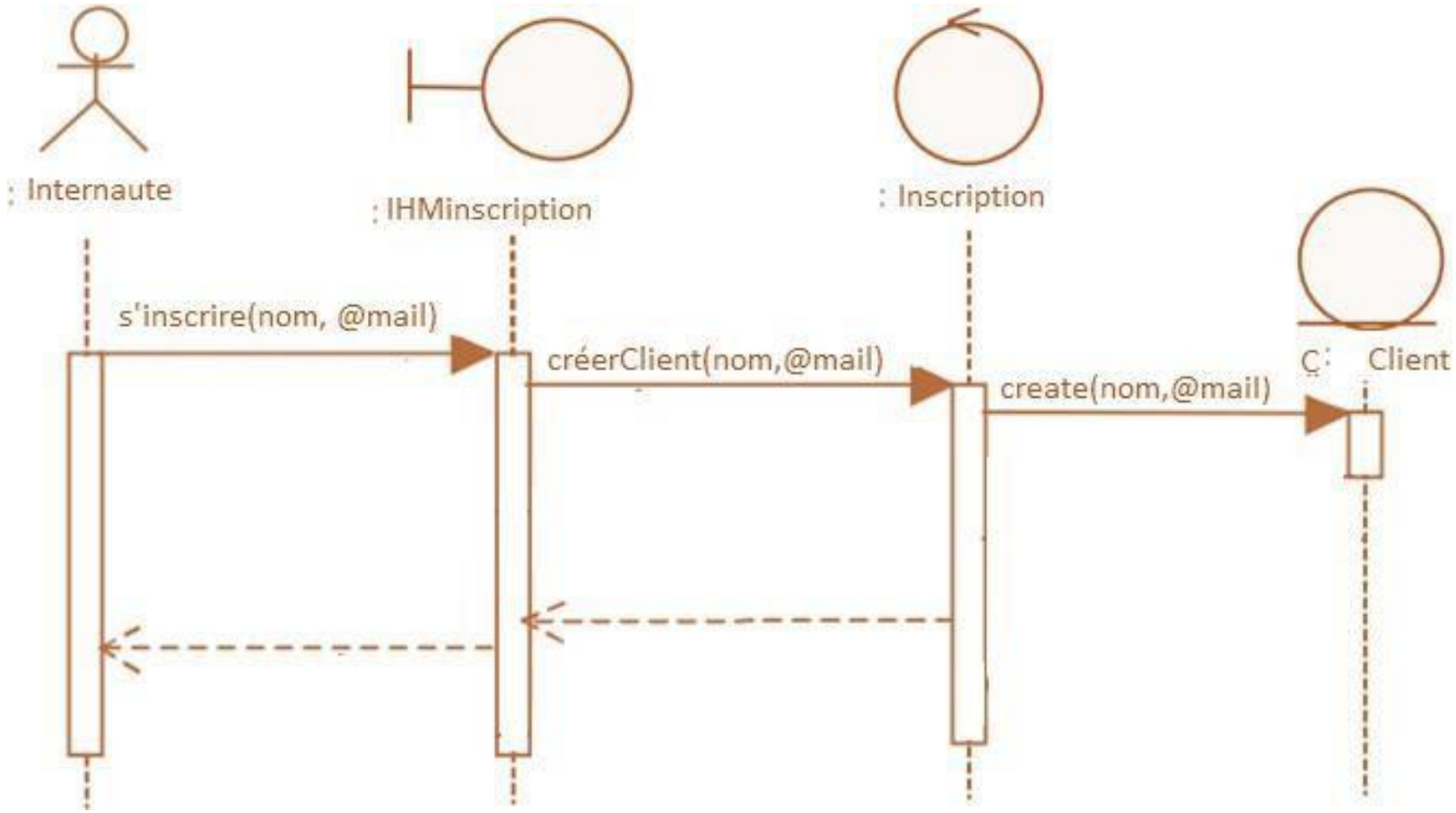


Diagramme de séquences: Je retiens

- 2 types: diagramme de séquences système (vision externe) et diagramme de séquences d'objets (vision interne)
- La notion de scénario
- Un message = appel d'une méthode de l'objet en général
- **DSQ D'objets:** on en fait autant qu'il y a de processus importants à détailler.
- **DSQ système:** utiles pour construire le dialogue H/M ou pour montrer les relations entre les composants physiques ou logiciels.

Exercice d'application: Etude de cas

Digramme de séquences: Le DAB

- Un cas d'utilisation est très souvent documenté par une description textuelle: Un diagramme de séquence permet de compléter et de visualiser simplement et intuitivement la description textuelle.

- Dans le cours sur le diagramme de cas d'utilisation, nous avons fait une description textuelle « orale » du cas d'utilisation « Retirer de l'argent » du DAB.

- Essayer de représenter le diagramme de séquence du scénario nominale.

Description des scénarios: Retirer de l'argent

Concept: Ce cas d'utilisation permet à un porteur de carte, qui n'est pas client de la banque, de retirer de l'argent, si son crédit hebdomadaire le permet.

Acteurs: Porteur de la carte (principal), Système d'autorisation (secondaire).

Pré conditions:

- La caisse du GAB est alimentée.
- Aucune carte ne se trouve déjà coincée dans le lecteur.
- La connexion avec le Système d'autorisation est opérationnelle.

Scénario nominal:

1 Le porteur de la carte introduit sa carte dans le lecteur de cartes du GAB.
2 Le GAB vérifie que la carte introduite est bien une carte bancaire.
3 Le GAB demande au porteur de carte de saisir son code d'identification.
4 Le porteur de carte saisit son code d'identification.
5 Le GAB compare le code d'identification avec celui qui est codé sur la puce de la carte.
6 Le GAB demande une autorisation au système d'autorisation
7 Le système d'autorisation donne son accord et indique le solde hebdomadaire.

8 Le GAB demande au porteur de carte de saisir le montant désiré du retrait.

9 Le porteur de la carte saisit le montant désiré du retrait.

10 Le GAB contrôle le montant demandé par rapport au solde hebdomadaire.

11 Le GAB demande au porteur de carte s'il veut un ticket.

12 Le porteur de carte demande un ticket.

13 Le GAB rend sa carte au porteur de carte.

14 Le porteur de carte reprend sa carte.

15 Le GAB délivre les billets et un ticket.

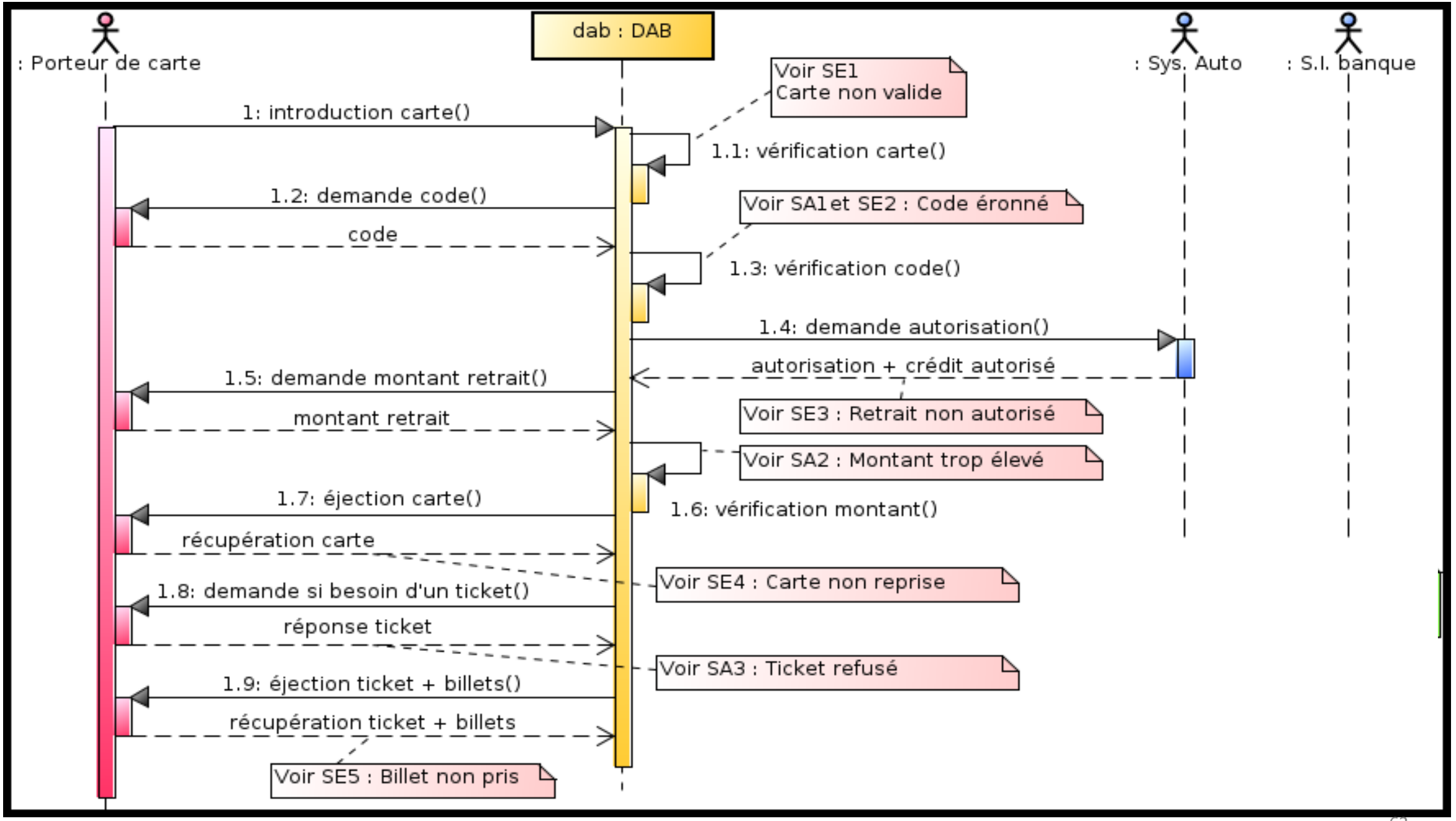
16 Le porteur de carte prend les billets et le ticket.

Solution:

Il suffit de transcrire sous forme de diagramme de séquence les interactions citées dans le scénario textuel précédent.

En utilisant les conventions graphiques propres au diagramme de séquence:

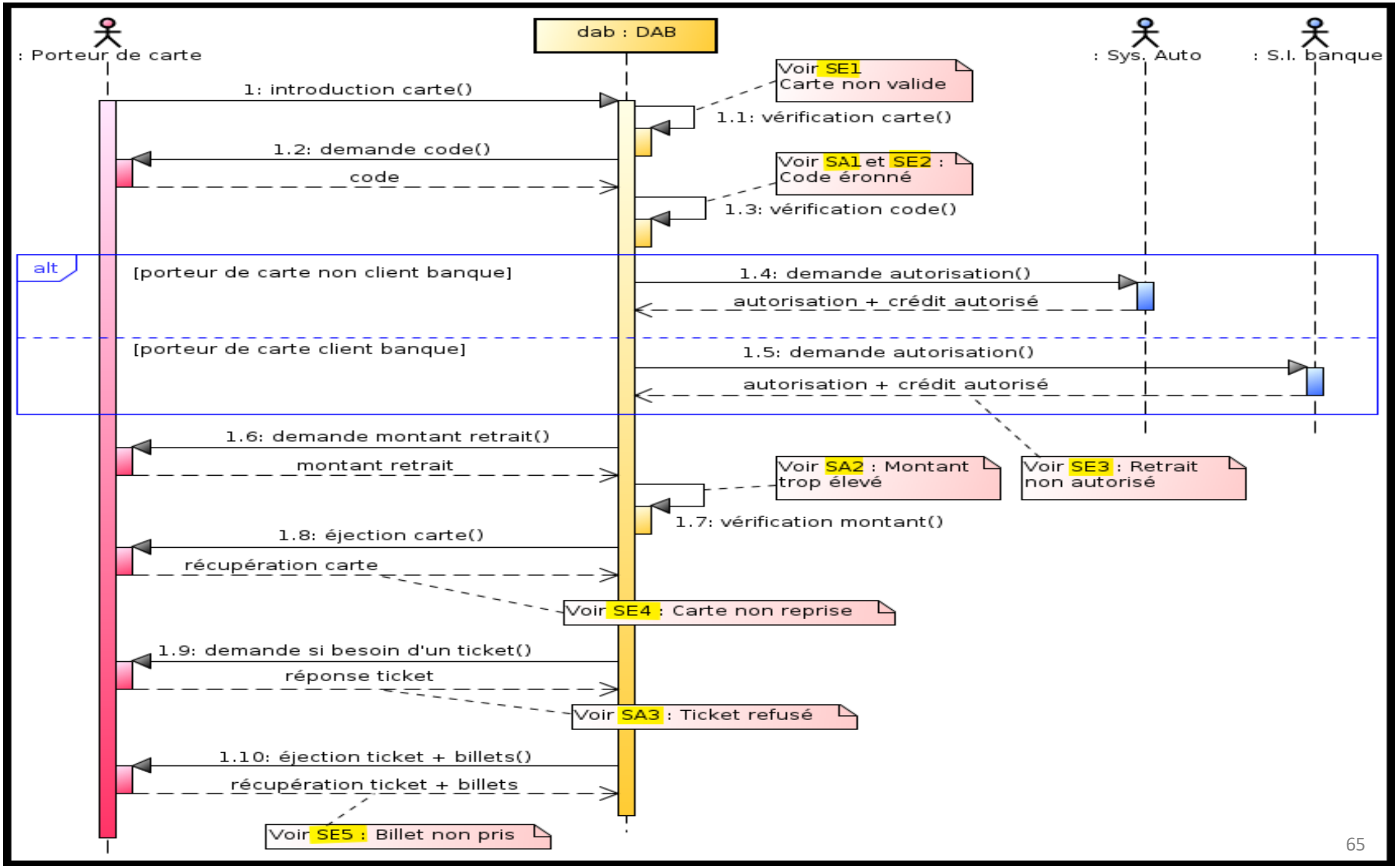
- L'acteur principal Porteur de carte à gauche ;
- Un participant représentant le GAB au milieu;
- L'acteur secondaire Système autorisation à droite du GAB.


: Porteur de carte
dab : DAB
: Sys. Auto
: S.I. banque
Voir SE1
Carte non valide
1: introduction carte()
1.1: vérification carte()
1.2: demande code()
Voir SA1 et SE2 : Code éronné
code
1.3: vérification code()
1.4: demande autorisation()
autorisation + crédit autorisé
1.5: demande montant retrait()
Voir SE3 : Retrait non autorisé
montant retrait
Voir SA2 : Montant trop élevé
1.6: vérification montant()
1.7: éjection carte()
récupération carte
Voir SE4 : Carte non reprise
1.8: demande si besoin d'un ticket()
réponse ticket
Voir SA3 : Ticket refusé
1.9: éjection ticket + billets()
récupération ticket + billets
Voir SE5 : Billet non pris

-Le diagramme de séquence ci-dessus ne correspond qu'au scénario nominal du cas d'utilisation.

-Des notes (optionnelles) permettent d'indiquer les scenarii alternatifs et d'exceptions. Chaque scénario nominal ou d'exception peut être documenté par son propre diagramme de séquence.

-Grâce aux fragments d'interactions, il est possible de documenter un scénario alternatif (ou d'exception) dans le diagramme de séquence du scénario nominale. Par exemple, le diagramme de séquence ci-dessous correspond à la description du scénario nominal et du scénario alternatif (porteur de carte client banque).


: Porteur de carte
dab : DAB
: Sys. Auto
: S.I. banque
1: introduction carte()
Voir SE1
Carte non valide
1.1: vérification carte()
1.2: demande code()
code
Voir SA1 et SE2 :
Code éronné
1.3: vérification code()
alt
[porteur de carte non client banque]
1.4: demande autorisation()
autorisation + crédit autorisé
[porteur de carte client banque]
1.5: demande autorisation()
autorisation + crédit autorisé
1.6: demande montant retrait()
montant retrait
Voir SA2 : Montant
trop élevé
Voir SE3 : Retrait
non autorisé
1.7: vérification montant()
1.8: éjection carte()
récupération carte
Voir SE4 : Carte non reprise
1.9: demande si besoin d'un ticket()
réponse ticket
Voir SA3 : Ticket refusé
1.10: éjection ticket + billets()
récupération ticket + billets
Voir SE5 : Billet non pris

Merci De Votre Attention